



廈門大學
XIAMEN UNIVERSITY

游戏设计 & 设计思维

线下研讨课-5

Tutorial-5

主讲：林俊聪

邮箱：jclin@xmu.edu.cn





contents

目录

• 技术前沿

- AI for Game
- Character AI
- Procedural Techniques
- Reinforcement Learning



廈門大學
XIAMEN UNIVERSITY

1 技術前沿

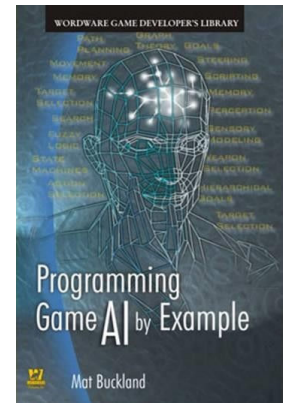
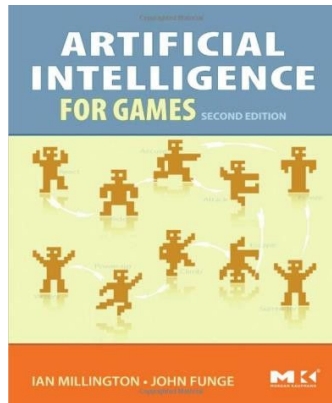
--自强不息，止于至善--



廈門大學
XIAMEN UNIVERSITY

2.1 AI for Game

--自强不息，止于至善--



“the question of whether computers can think is like the question of whether submarines can swim” -- Dijkstra

Game AI: The set of algorithms, representations, tools, and tricks that support the creation and management of real-time digital experiences

Goals of AI



Systems that think like humans	Systems that think rationally
Systems that act like humans	Systems that act rationally

Artificial Intelligence



- Getting a computer to do something that a “reasonable person” would think requires intelligence
 - Complexity fallacy
 - Simple can be good, Complex can be bad.
 - Illusion of complexity & Illusion of intelligence: Behaviorism
 - M&F: “We are not interested in the nature of reality or mind; we want characters that look right. In most cases, this means starting from human behaviors and trying to work out the easiest way to implement them in software.”
- Automation

Artificial Intelligence for Games



廈門大學
XIAMEN UNIVERSITY

- What is different
 - Gaming stresses computing resources
 - Graphics Engine
 - Physics Engine
 - Updates at fixed frame rates
 - Need to keep AI simple or use a complicated time allocation scheme
 - AI needs to look good, but not be good
 - Human beings are likely to attribute intelligent behavior to inanimate objects whose functioning they do not understand

Why AI in games?



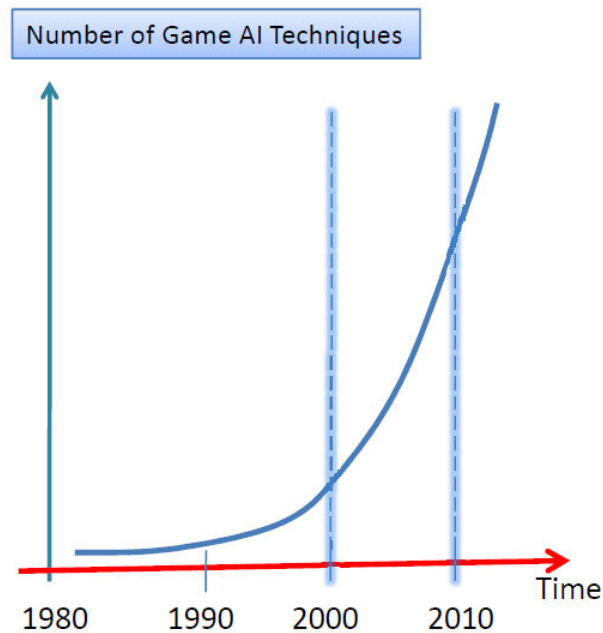
- Automation—because you need other people to do things, but don't always have those people
 - Opponents
 - Companions
 - NPCs (shopkeepers, farmers, villains)
 - Dungeon master?
 - Plot writer?
 - Game designer?

Why AI is important for games

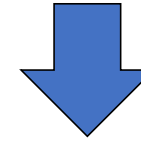


- Why is it essential to the modeled world?
 - NPC's of all types: opponents, helpers, extras, ...
- How can it hurt?
 - Unrealistic characters → reduced immersion
 - Stupid, lame behaviors → reduced fun
 - Superhuman behaviors → reduced fun
- Until recently, given short shrift by developers. Why?
 - Graphics ate almost all the resources
 - Can't start developing the AI until the modeled world was ready to run
 - AI development always late in development cycle
- Situation rapidly changing / changed. How?
 - AI now viewed as helpful in selling the product
 - Still one of the key constraints on game design

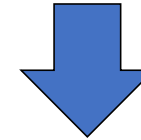
Development of Game AI



Field of game AI has been evolving and expanding.



This rapid changing makes a gap between academic AI research and AI development in Game Industry.



But game developers had not passed their research themes to academic researchers enough.

Development of Game AI



Game AI Industry Conference/Community

GDC AI Summit
(Game Developer's
Conference)

<http://www.gdconf.com/>

Paris Game AI Conference

<http://gameaiconf.com/>

AI Game Dev.COM

<http://aigamedev.com/>

AI Game Programmers Guild

<http://www.gameai.com/>

Game AI Academic Conference/Community

AIIDE
(AI and Interactive
Digital Entertainment)

<http://gameaiconf.com/>

IEEE CIG
(IEEE Computational
Intelligence and Games)

<http://aigamedev.com/>

IEEE CIS
(IEEE Computational
Intelligence Society)

<http://www.gameai.com/>

How/Why distinct from “academic AI”



- Supporting the player experience
- Good game AI == matching right behaviors to right algorithms
- Product is the target, not clever coding – ends justify means. FUN
- Illusion of intelligence
- “Magic Circle” (Rules of play: game design fundamentals)
- Elegance in simplicity & the complexity fallacy
- Quality control & resource limits
- Fun vs smart: goal is not always to beat the player
- Optimal/rational is rarely the right thing to do

Common (game) “AI” Tricks?



- Move before firing – no cheap shots
- Be visible
- Have horrible aim (being Rambo is fun)
- Miss the first time
- Warn the player
- Attack “kung fu” style (Fist of Fury; BL vs School)
- Tell the player what you are doing (especially companions)
- React to own mistakes
- Pull back at the last minute
- Intentional vulnerabilities or predictable patterns

Common Heuristics

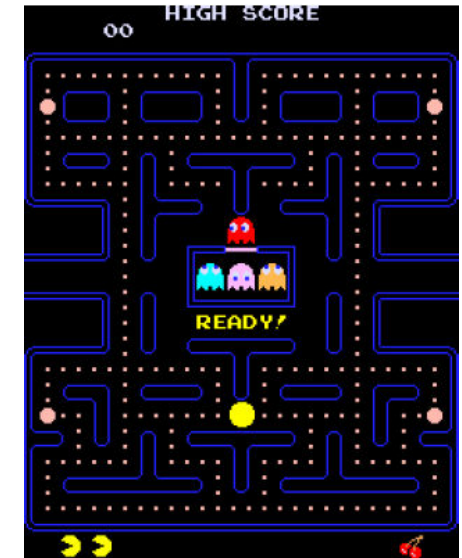
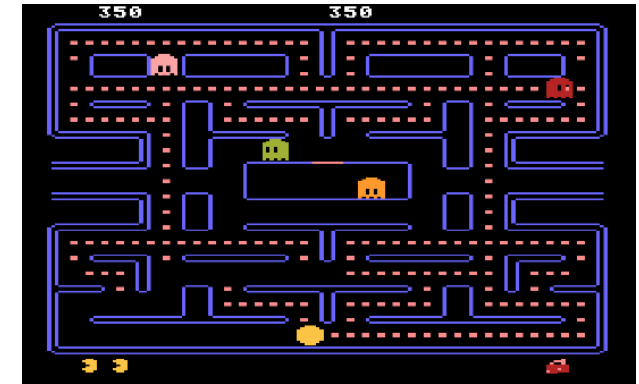


- Most Constrained / Most Difficult
 - Given current state make choice / select action least available.
 - Aka: Do the most difficult thing first
 - E.g. Attacking enemy with special weapon
 - E.g. Assigning large characters to squads first
- Try the most promising thing first
 - Give each option a rough score and attempt in decreasing order

Attribution of Intelligence



- Simple things can look good
 - Pac-Man 1979
 - Uses a simple state machine for the behavior of ghosts
 - Each of the four ghosts moves in a straight line
 - At a junction, they choose a route
 - With a ghost-dependent likelihood:
 - Go in the direction of the player
 - Based on coordinates, no path finding
 - Take a random route
 - Users attribute intelligence to the ghosts



Attribution of Intelligence



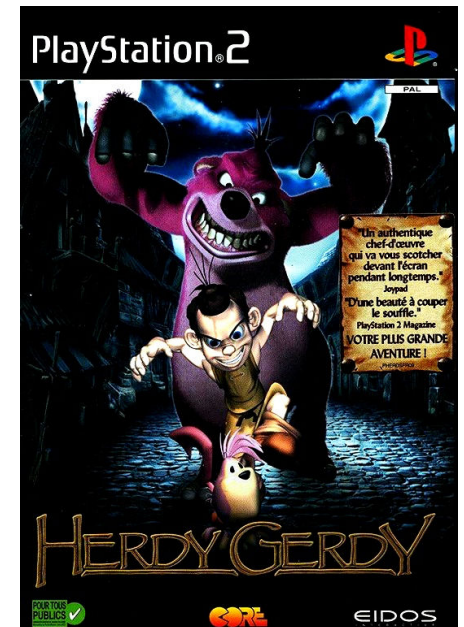
- Simple things can look good
 - Pac-Man 1979
 - Shooter game:
 - Enemy avatars home in on player
 - Randomly make a side step
 - Tester feed-back
 - Appear to be anticipating shoots and save themselves

Attribution of Intelligence

- Complex things can look bad
 - Herdy Gerdy” 2002 for PS2
 - "Herdy Gerdy aims high and delivers little, easily placing it on the list of the PS2's worst games."
 - Herdy Gerdy had herding as its concept
 - Movement AI has problems
 - Characters get caught in scenery
 - Collision detection leads animals into areas where they get stuck

<https://www.gamespot.com/reviews/herdy-gerdy-review/1900-2857230/>

One of the primary issues with the gameplay is the creature AI. Doops are stupid, and there's always one stray that seems to want to ignore the rest of the pack and do its own thing. This can cause headaches, particularly in the timed boss levels when you're trying to win an item that will give Gerdy new abilities. Creatures will also get stuck on portions of the level and remain stuck until the game is reset. Another problem is that each group of creatures seems predetermined to be herded to a particular pen. You may have a group of 15 doops right in front of a doop pen, but they will fight going in and instead run in the opposite direction toward another doop pen. Because of these AI problems, it's always good to keep the camera panned way out, but this only brings the game's largest issue to the fore.



Attribution of Intelligence

- Complex things can look bad
 - Black and White (2001)
 - Successful game where players have to teach pets
 - Possible to teach bad habits that cannot be unlearned
 - Pets are incapable of performing simple tasks

<https://www.gamespot.com/reviews/black-and-white-review/1900-2702884/>

In spite of all this, most players will agree that the giant creatures are the best thing about Black & White--each god commands one as its personal avatar. You can teach the creature to mimic pretty much any of your own godly interventions. The game's sophisticated behavioral artificial intelligence makes it fascinating and very exciting to watch the creature learn any number of these things. The good news is that this aspect of the game--even though it's secondary to the core strategy game in Black & White--is even more satisfying and at least as involved. You gain your choice of creature after about a half hour of play. It starts off with the mind of a baby; it'll wander around, clueless but curious. As its curiosity inspires it to try many different things, you simply gesture with the mouse to give it positive reinforcement (by petting it lightly) or negative reinforcement (with a slap, or even a beating), depending on whether you want it to act that way again. You can guide where it moves by using one of several special leashes, which can also influence it to learn more actively, to act more kindly, or to act more aggressively. Creatures will also begin to learn miracles as you cast them in their vicinity. Soon, you can turn your giant cow (or ape, or tiger) into a farmhand by teaching it to water your crops and to toss food and wood into your stockpile. You'll also have to train it to eat a certain diet--you can pretty much convince it to try anything at least once, which can be very funny--and you'll even have to encourage it not to relieve itself on your temple.



Attribution of Intelligence



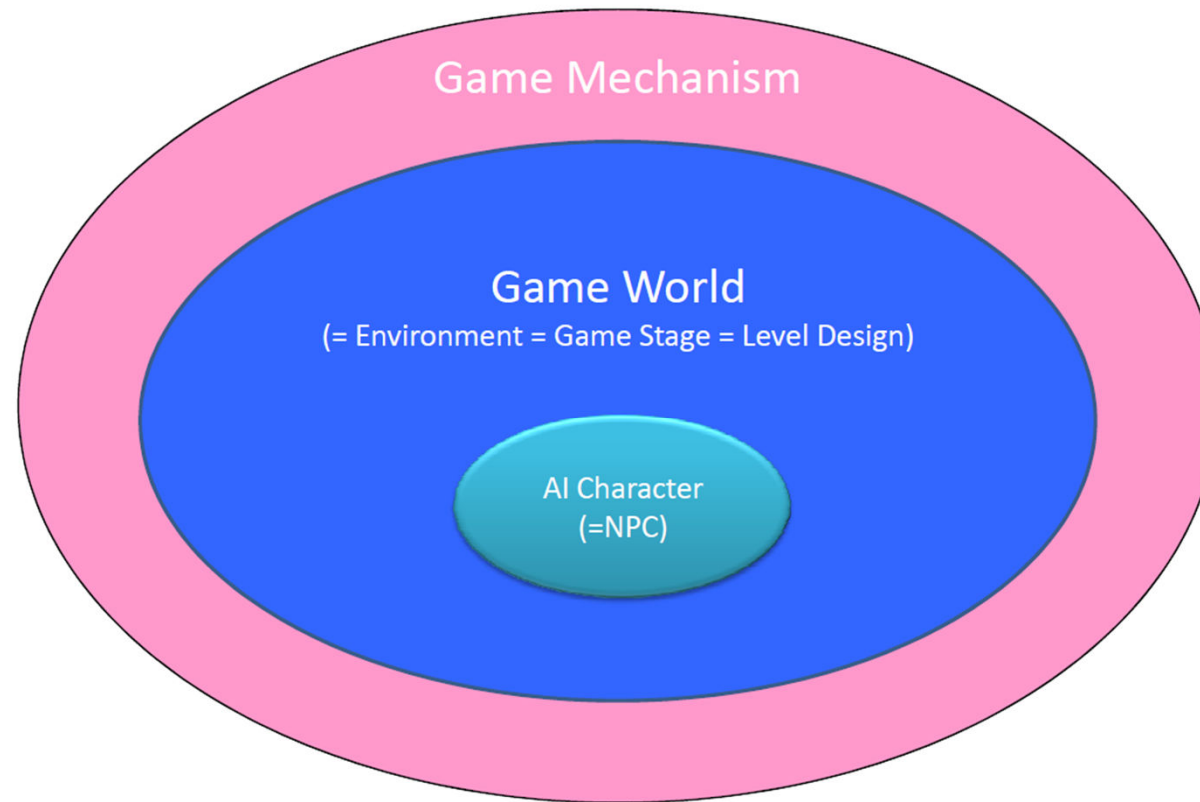
- Perception Window
 - Most interaction with avatars is for very short time
 - Simple AI can be better
 - Complex AI can be interpreted as stupid
 - Example:
 - Modeling a guard that has to be “taken out”
 - Guard hears noise, turns away and goes to a corner
 - Appears stupid
 - Guard wants to turn on light
 - Seems a good idea
 - Guard will be shot dead before the player can see that his behavior is reasonably intelligent

Attribution of Intelligence

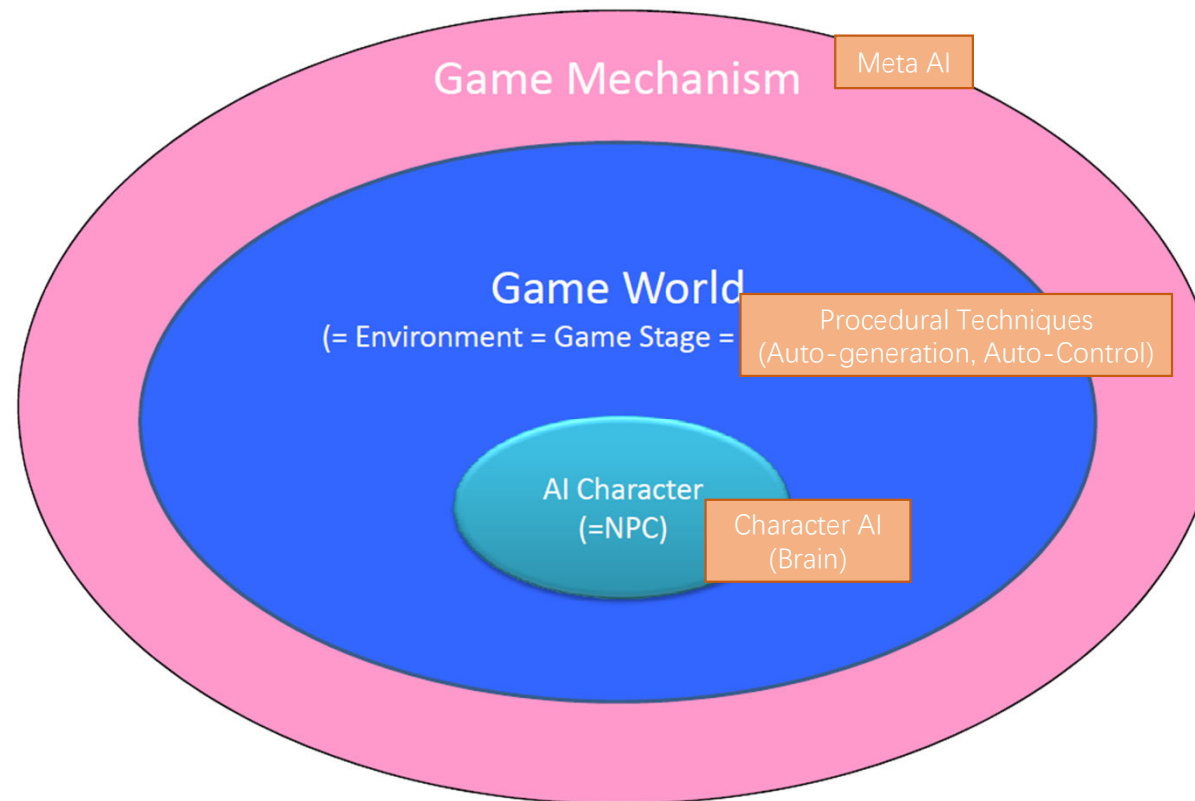


- Perception Window
 - Changes in behavior are more obvious to players than behavior itself
 - WoW:
 - Monsters do not change behavior when players start killing them

AI in Game World



AI in Game World

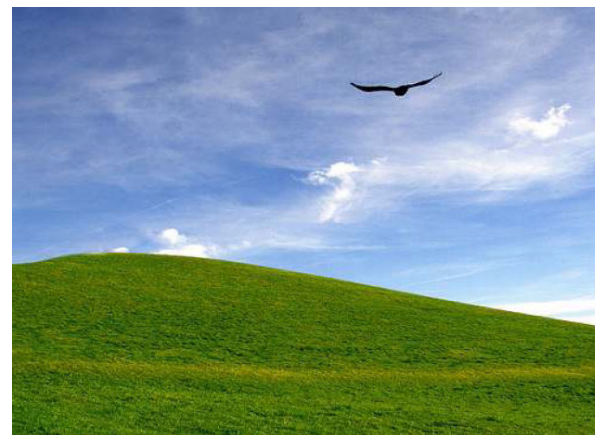




廈門大學
XIAMEN UNIVERSITY

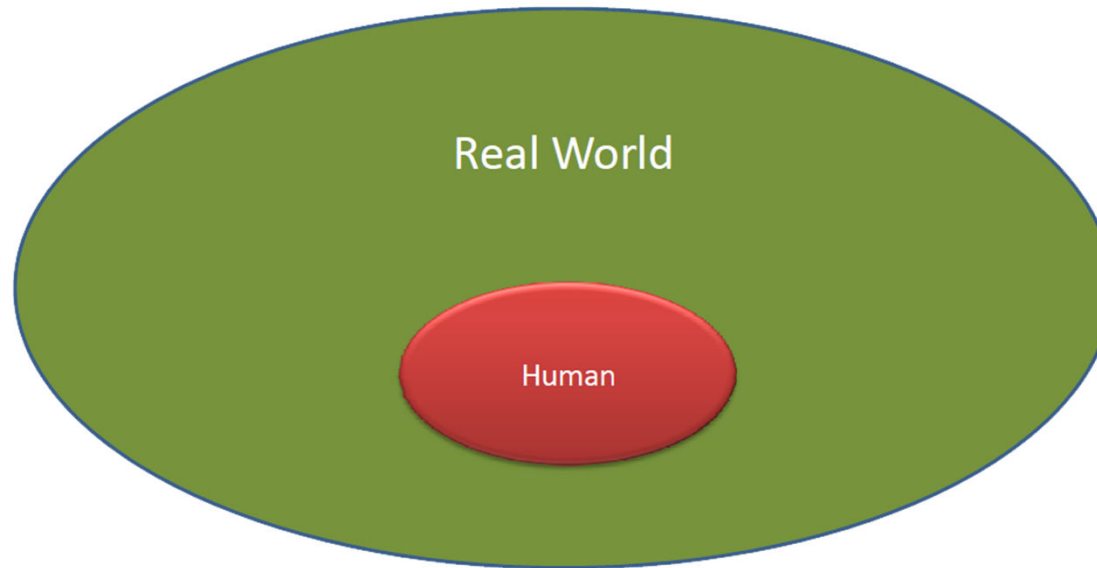
2.2 Character AI

--自强不息，止于至善--



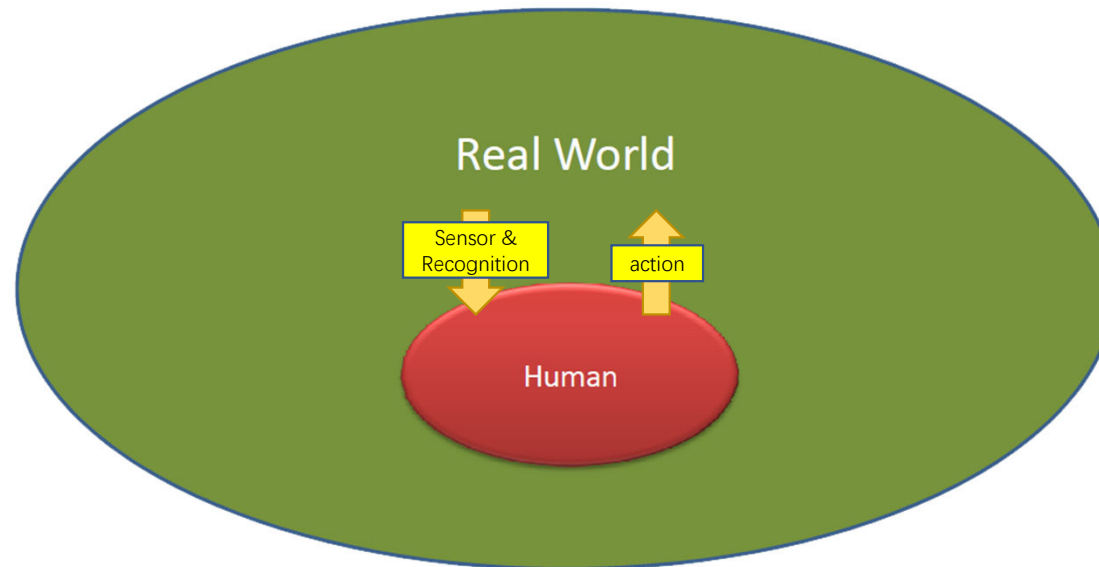
自强不息，止于至善

Life is in the World



Human lives in Real World.

Life is in the World



Human lives in Real World.

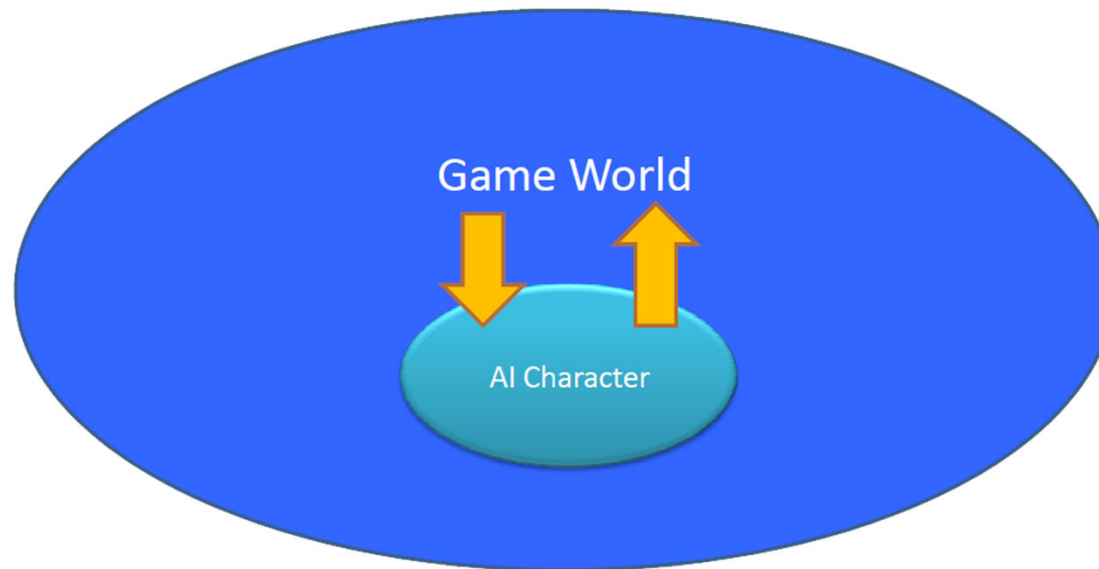
Human and Environment interact each other.

AI is in the Game World



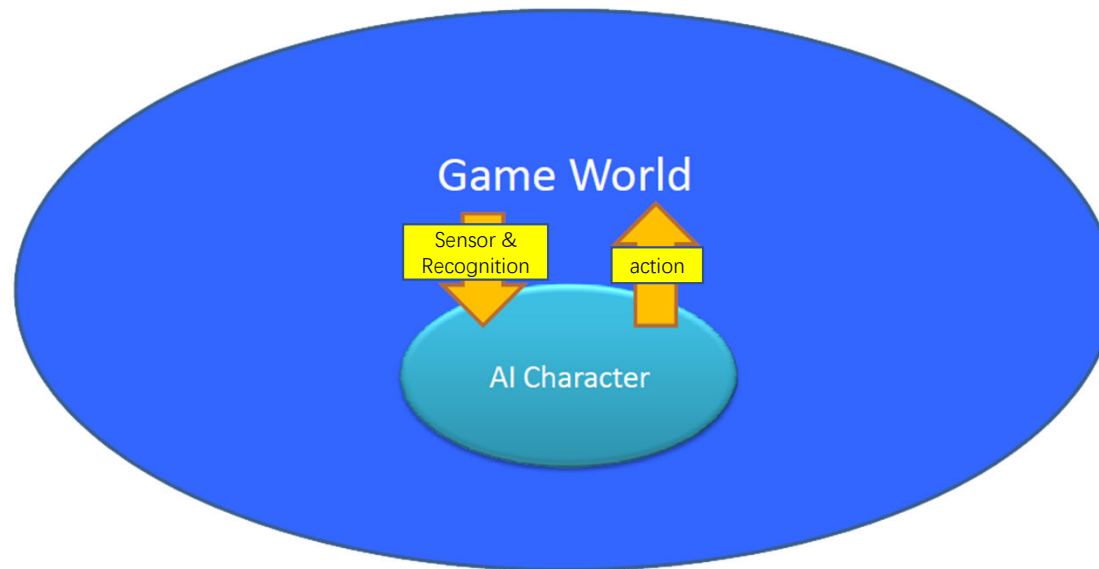
自强不息，止于至善

AI is in the Game World



AI lives in Game World.

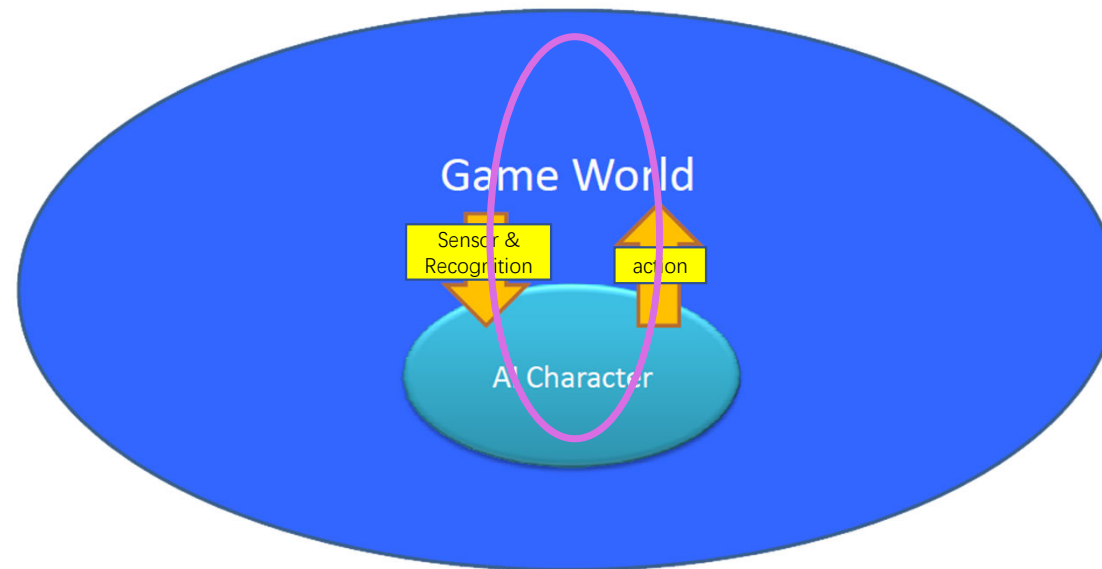
AI is in the Game World



AI lives in Game World.

AI and Environment interact each other.

AI is in the Game World

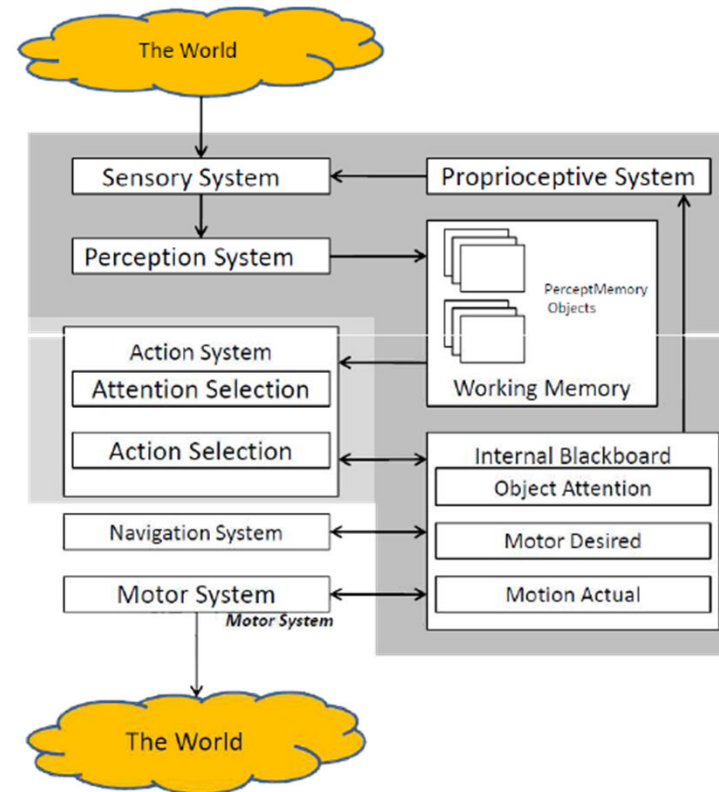


It makes “Information Flow” between AI and Environment

C4 Architecture



MIT Media Lab.
Synthetic Characters Group
Researching Virtual Pet in Digital World.

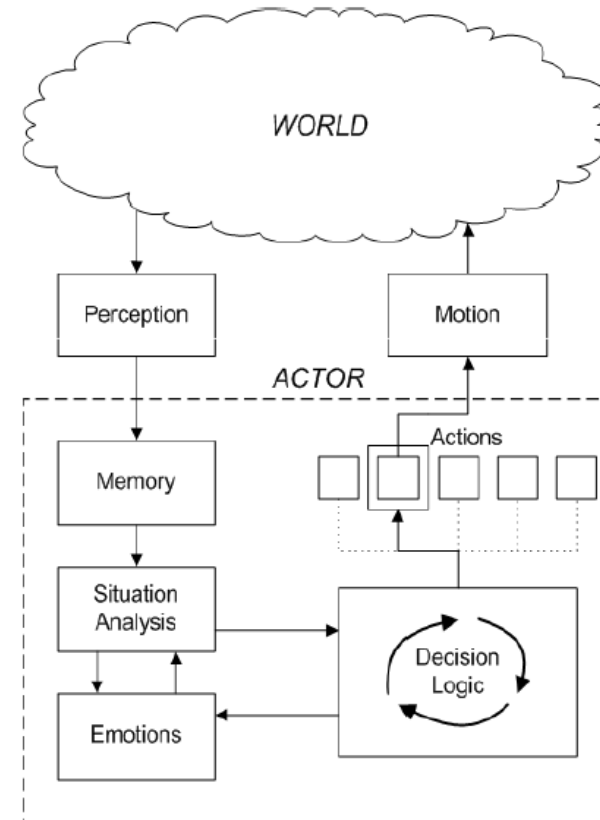


D. Isla, R. Burke, M. Downie, B. Blumberg (2001).,
"A Layered Brain Architecture for Synthetic Creatures", <http://characters.media.mit.edu/Papers/ijcai01.pdf>

Halo Agent Architecture

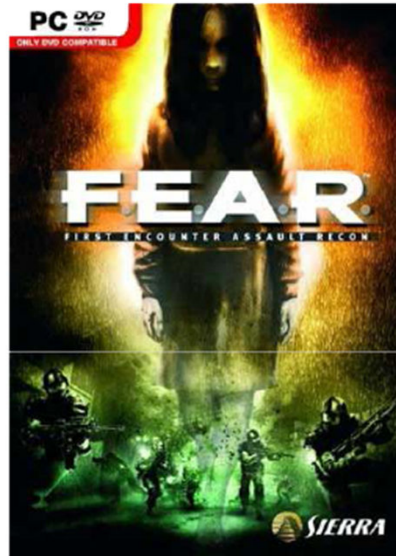


- ◆ Genre : SciFi-FPS
- ◆ Developer: BUNGIE Studio
- ◆ Publisher : Microsoft
- ◆ Hardware: Xbox, Windows, Mac
- ◆ Year: 2002

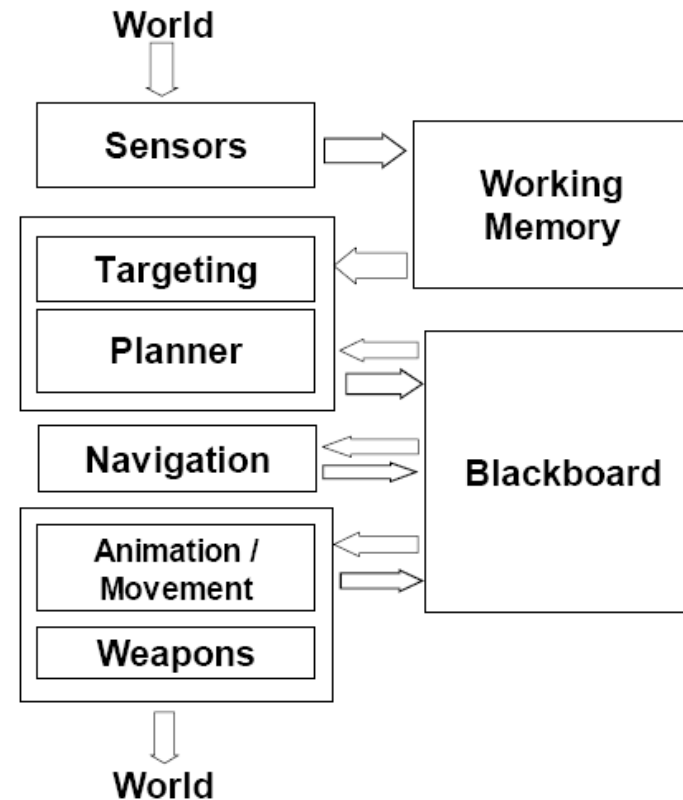


Jaime Griesemer(GDC 2002),The Illusion of Intelligence: The Integration of AI and Level Design in Halo
<http://www.bungie.net/Inside/publications.aspx>

Halo Agent Architecture



- ◆ Genre: Horror FPS
- ◆ Developer: Monolith Production
- ◆ Publisher: SIERRA
- ◆ Hardware: Windows
- ◆ Year: 2004

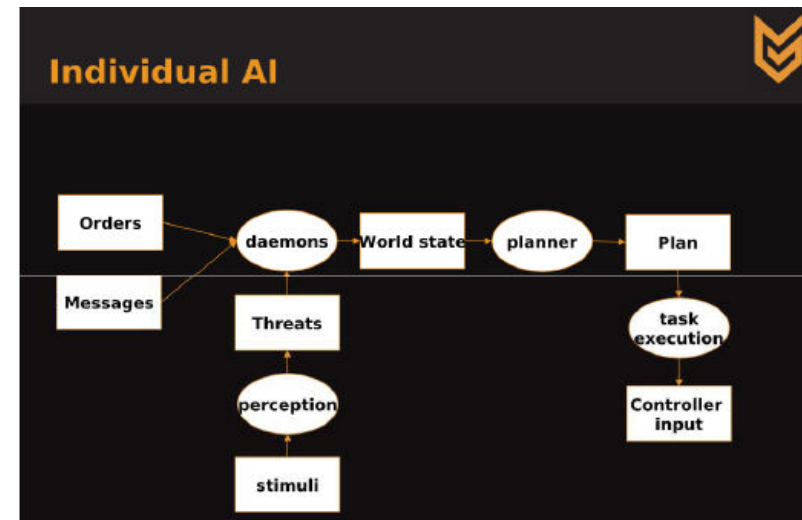


Agent Architecture Considerations for Real-Time Planning in Games (AIIDE 2005)
http://web.media.mit.edu/~jorkin/AIIDE05_Orkin_Planning.ppt

Halo Agent Architecture

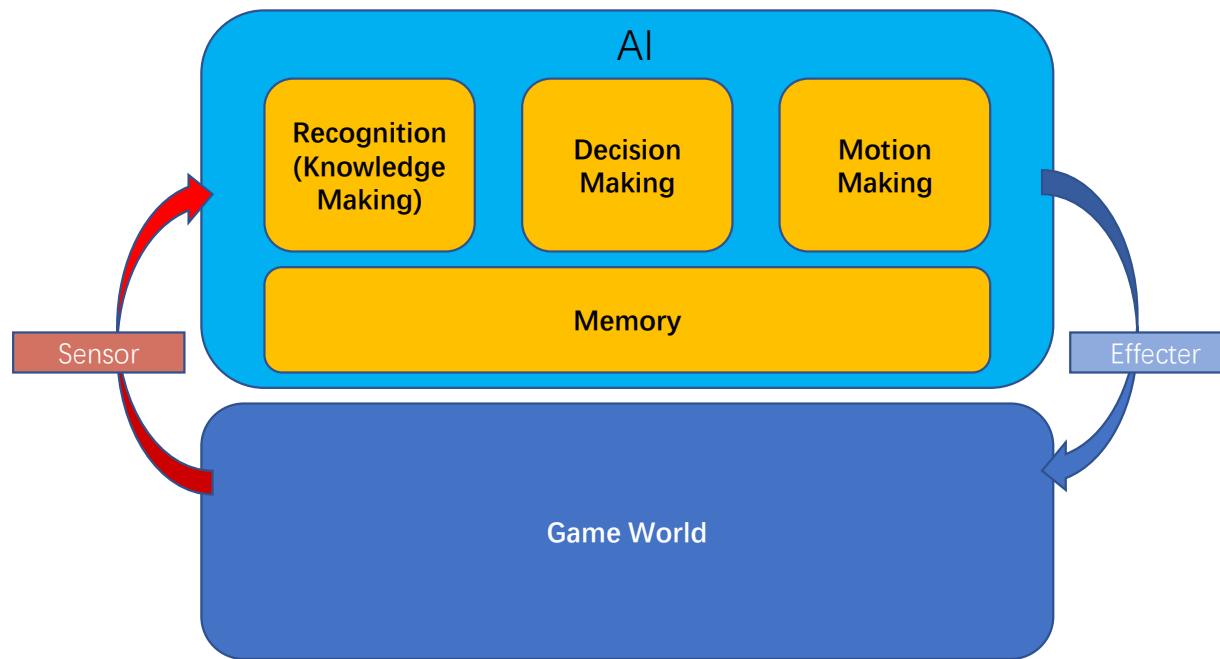


- ◆ Genre: FPS
- ◆ Developer: Guerrilla Games
- ◆ Publisher : SCE
- ◆ Hardware: PlayStation 3
- ◆ Year: 2009

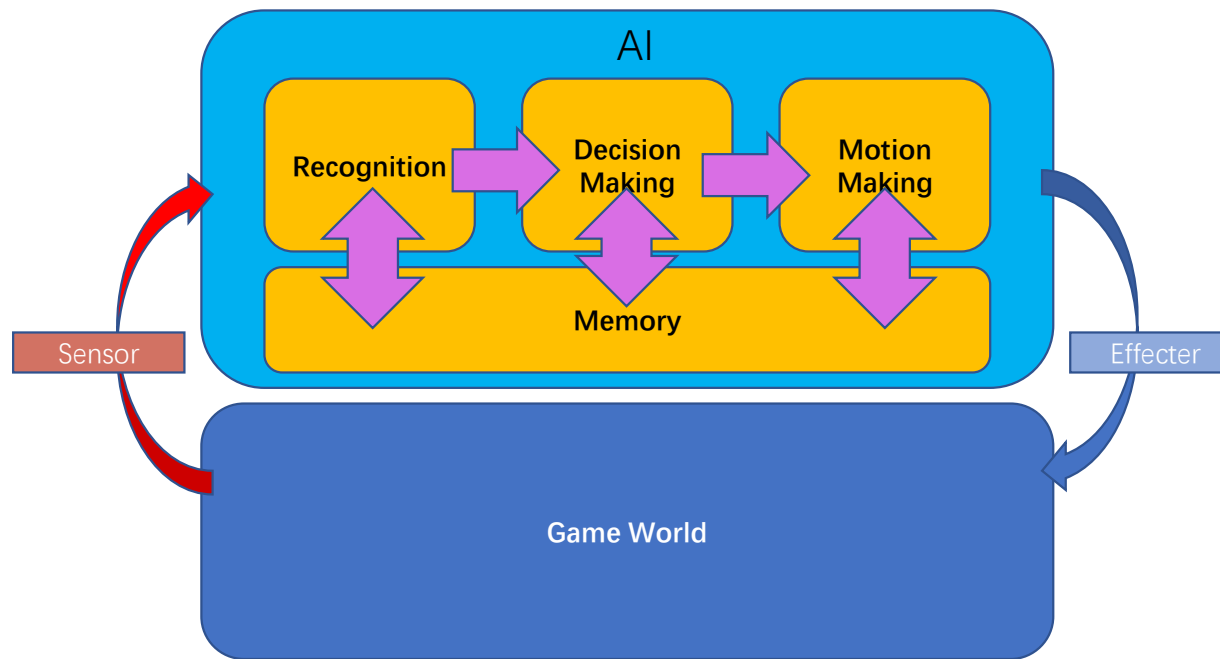


Based on: Alex Champandard, Tim Verweij, Remco Straatman, "Killzone 2 Multiplayer Bots",
http://files.aigamedev.com/coverage/GAIC09_Killzone2Bots_StraatmanChampandard.pdf

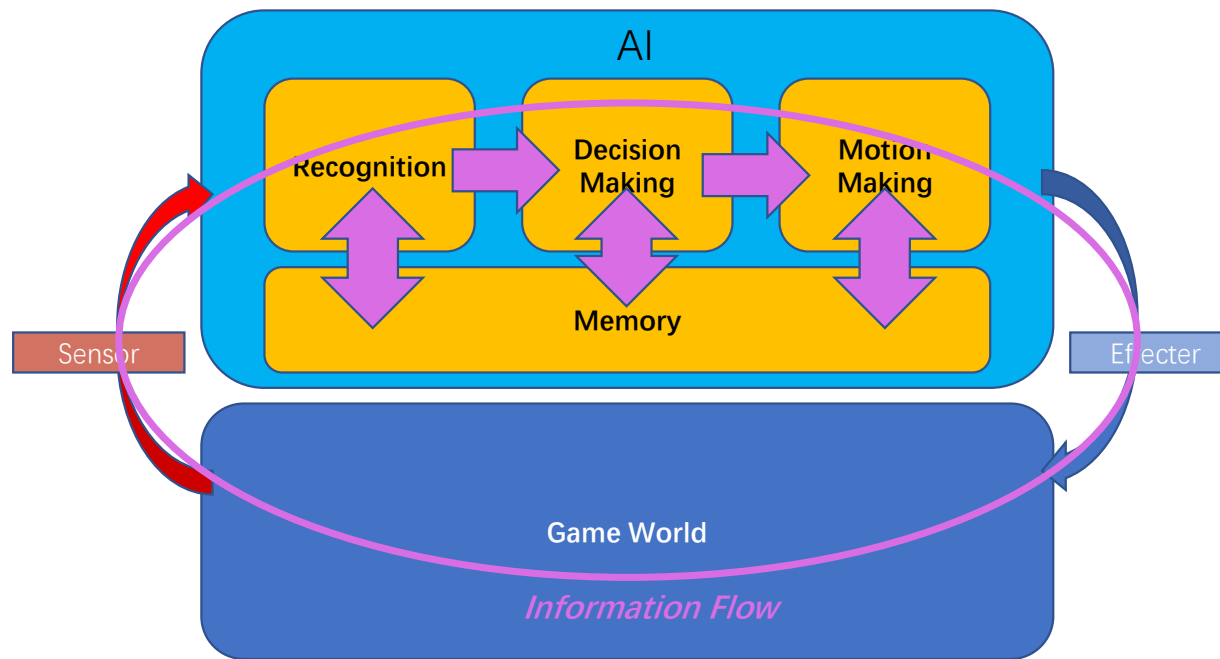
Agent Architecture



Agent Architecture



Agent Architecture



Character AI Research themes



- Agent Architecture (Framework)
- Sensor
- Recognition
- Memory
- Decision Making
- Motion Making
- ...

Character AI Research themes



- Agent Architecture (Framework)

- Sensor



- Recognition



- Memory

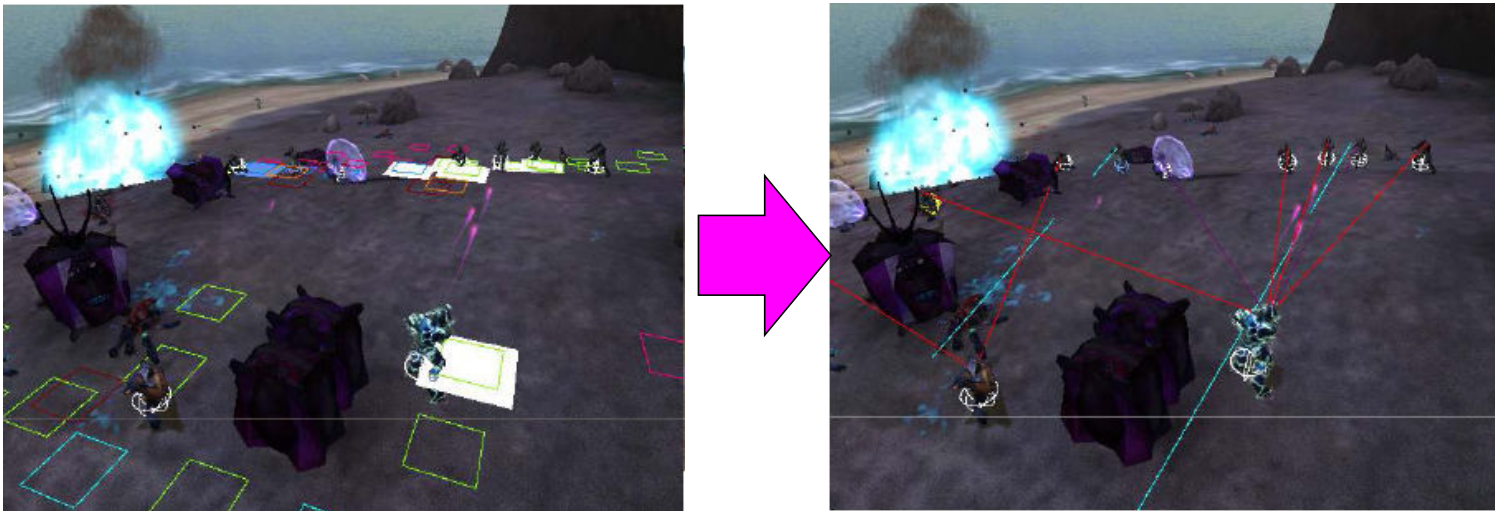
Knowledge Making

- Decision Making

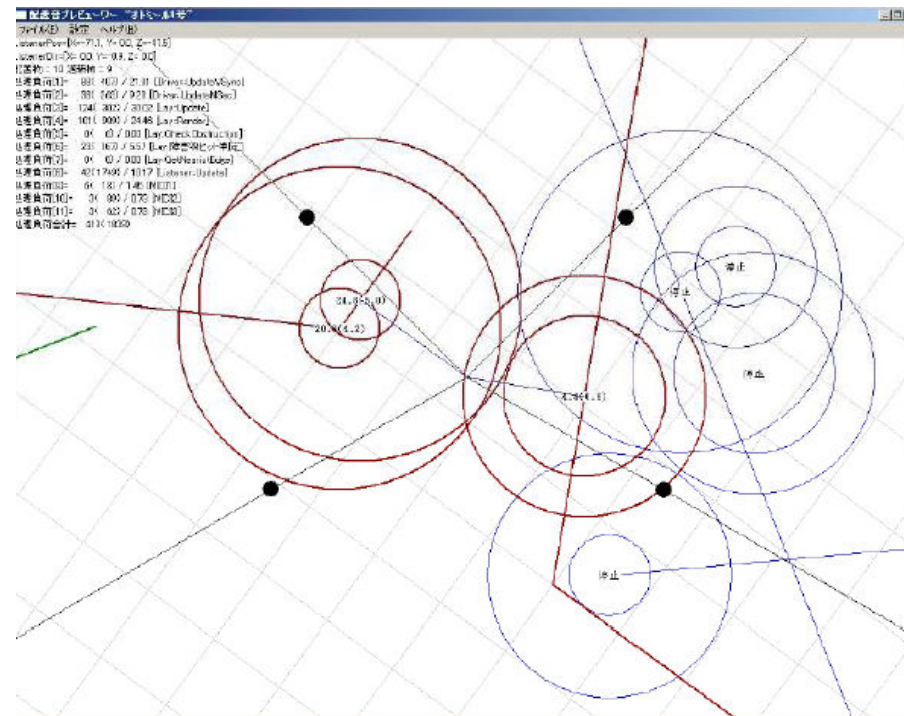
- Motion Making

- ...

Sensor



In many cases, Visual Sensor is implemented as ray-casting model.

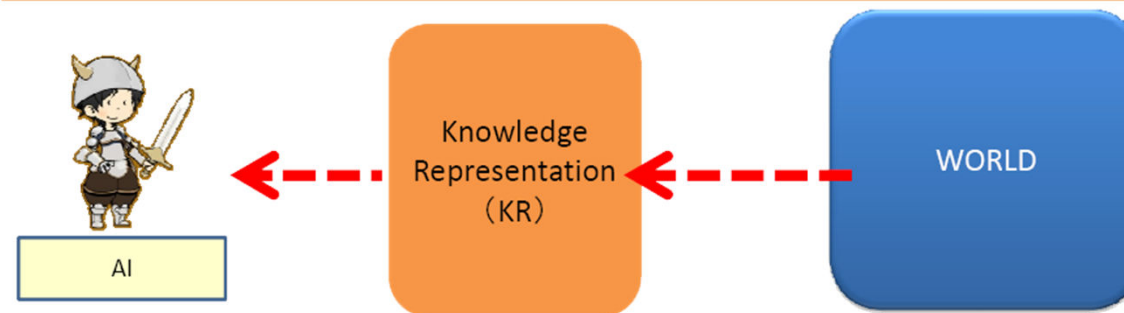


In many cases, Hearing Sensor is implemented as volume model.

Knowledge Representation



Can AI recognize sufficiently the world by his sensors ? **No.**



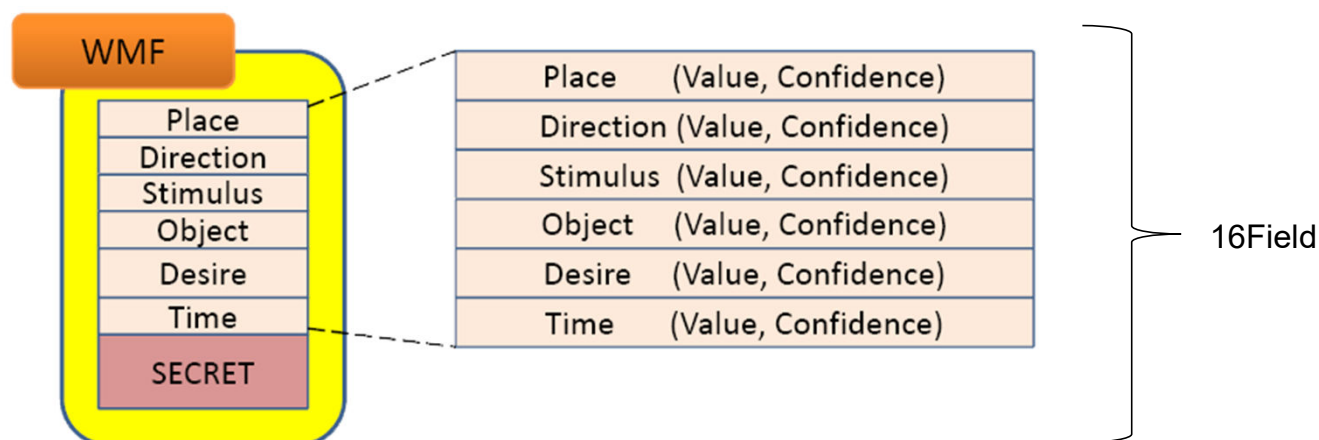
Information to help AI recognize the World (Object/Fact/Space)
= **Knowledge Representation(KR)**

Knowledge Representation

Working Memory Fact (WMF)

In F.E.A.R

Character, Object, Mission, Fact, Path, Node, Desire is presented as Unified Form “WMF”



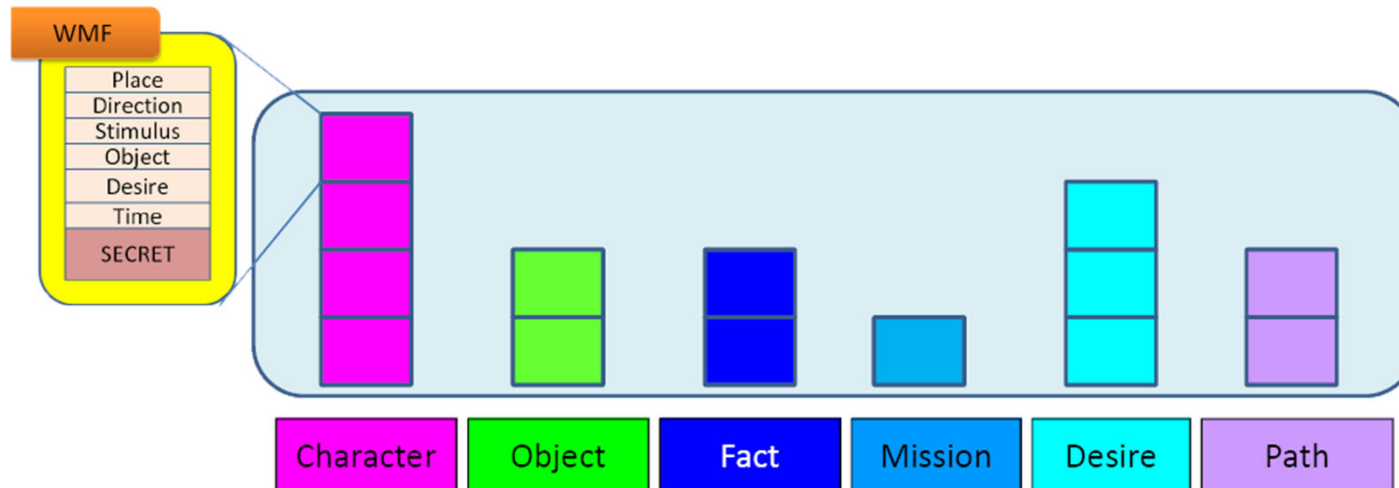
```
WorkingMemoryFact
{
    Attribute<Vector3D>    Position
    Attribute<Vector3D>    Direction
    Attribute<StimulusType> Stimulus
    Attribute<Handle>      Object
    Attribute<float>       Desire
    ...
    float fUpdateTime
}
```

```
Attribute<Type>
{
    Type Value
    float fConfidence
}
```

Value
Confidence

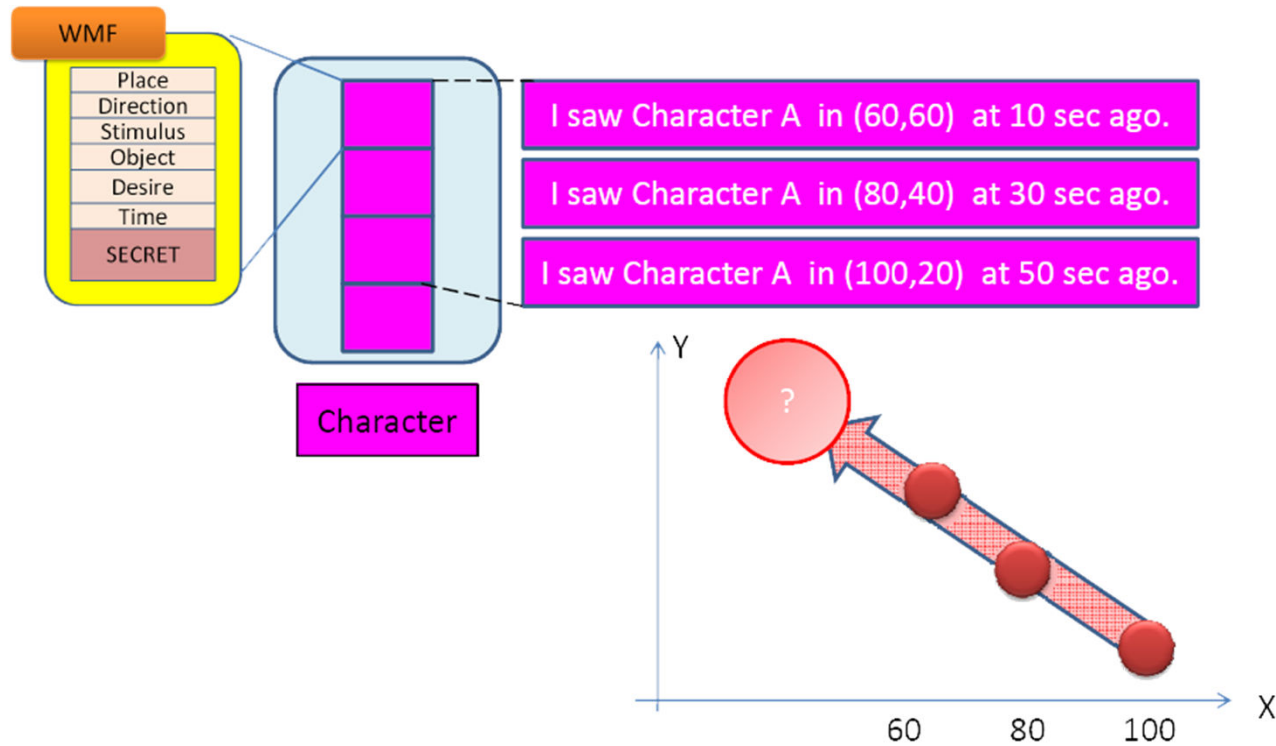
WMF Stacks

Facts AI recognized are stacked in Working Memory as WMFs in each field.



WMF Stacks

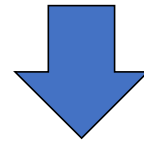
Facts AI recognized are stacked in Working Memory as WMFs in each field.
By using WMF stack, AI can estimate some knowledge of the future.



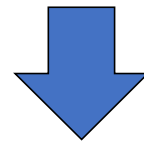
Knowledge Representation (KR)



How many kinds of KR are there ?



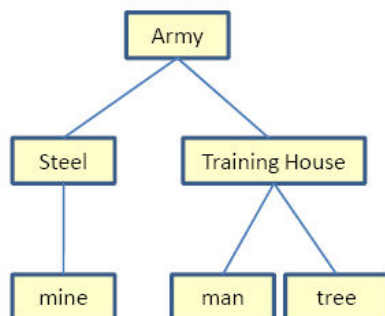
Number of how AI should recognize a fact.



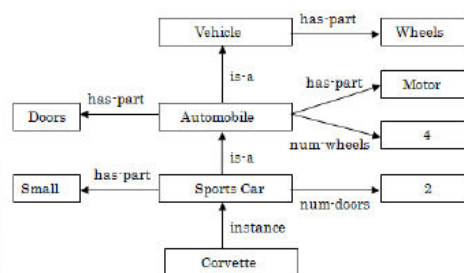
“KR”s is the base to make high intelligence

Variation of Knowledge Representation

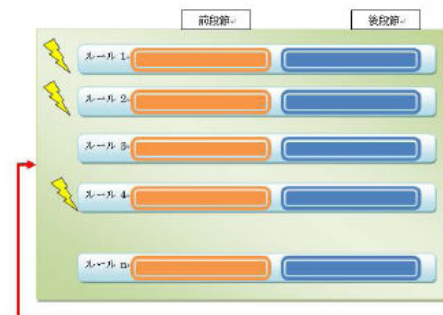
Dependency Graph



Semantic Network



Rule-based Representation



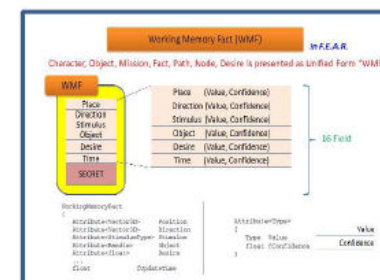
World Representation



Enemy Representation



WMF



World Representation (one of KR)

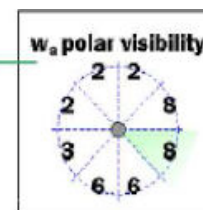
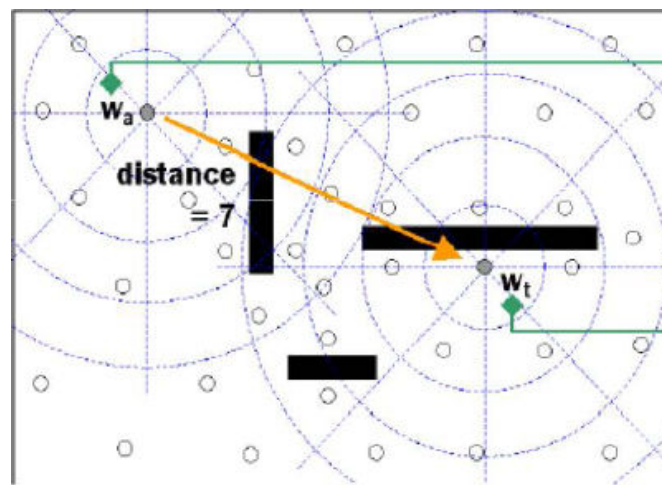


廈門大學
XIAMEN UNIVERSITY

World Representation (WR)
= KR for Game Stage
(Representation based on Waypoint or Navigation Mesh)

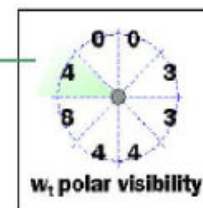


- ◆ FPS
- ◆ Guerrilla
- ◆ SCEE/SEGA
- ◆ PS2
- ◆ 2004/2005



w_a may have line-of-fire to w_t if:

- $\text{table}(w_a, d) \geq 7$ and
- $\text{table}(w_t, d'') \geq 7$



w_t has cover from w_a if:

- $\text{table}(w_t, d'') < 7$ or
- $\text{table}(w_a, d) < 7$

Waypoints with 8-direction LOS(visible distance) (Killzone)

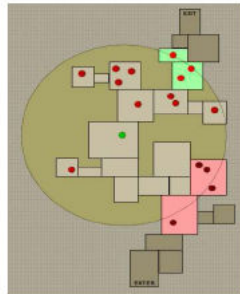
Straatman, R., Beij, A., Sterren, W.V.D., "Killzone's AI : Dynamic Procedural Combat Tactics", 2005
http://www.cgf-ai.com/docs/straatman_remco_killzone_ai.pdf

Variation of Knowledge Representation



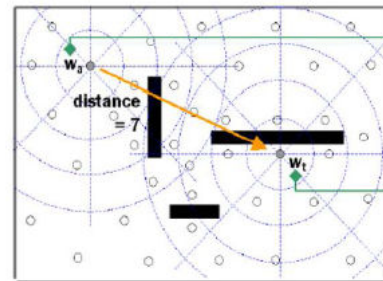
廈門大學
XIAMEN UNIVERSITY

Distribution Map



Left 4 Dead

Waypoints with LOS



Killzone

Territory Representation



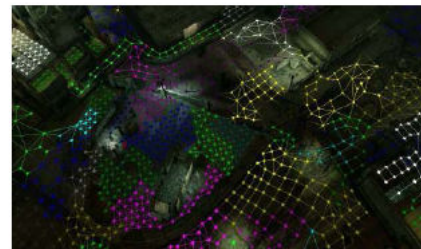
Halo2

Tactical Point System



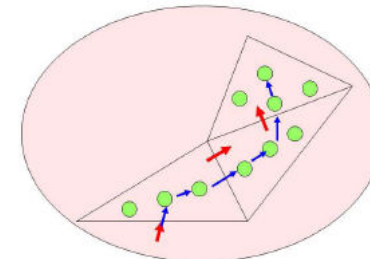
Halo

Tactical Map Clustering



Killzone2

NavMesh-Waypoint
Hierarchical Representation



Assassin's Creed

World Representation (one of KR)

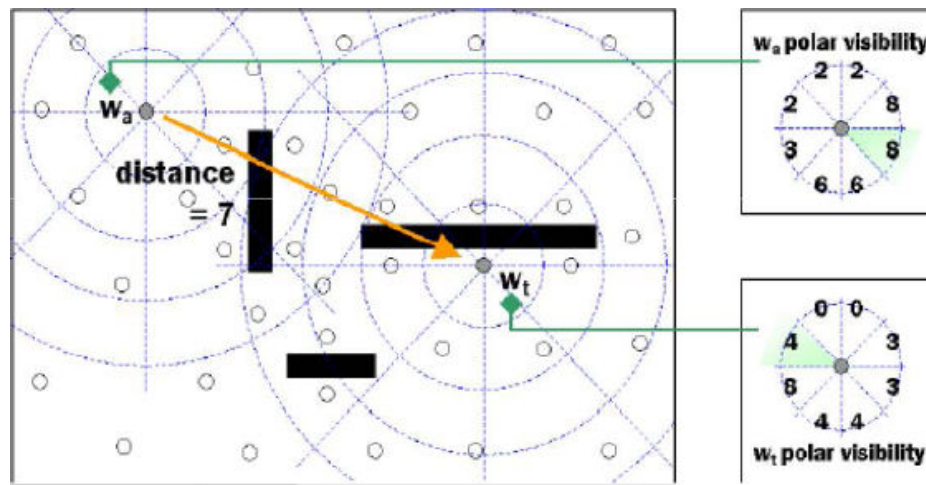


廈門大學
XIAMEN UNIVERSITY

World Representation (WR)
= KR for Game Stage
(Representation based on Waypoint or Navigation Mesh)



- ◆ FPS
- ◆ Guerrilla
- ◆ SCEE/SEGA
- ◆ PS2
- ◆ 2004/2005



w_a may have line-of-fire to w_t if:

- $\text{table}(w_a, d) \geq 7$ and
- $\text{table}(w_t, d'') \geq 7$

w_t has cover from w_a if:

- $\text{table}(w_t, d'') < 7$ or
- $\text{table}(w_a, d) < 7$

Waypoints with 8-direction LOS(visible distance) (Killzone)

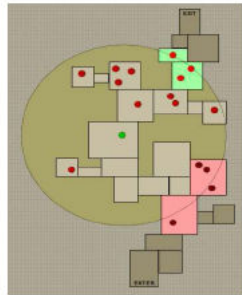
Straatman, R., Beij, A., Sterren, W.V.D., "Killzone's AI : Dynamic Procedural Combat Tactics", 2005
http://www.cgf-ai.com/docs/straatman_remco_killzone_ai.pdf

Variation of Knowledge Representation



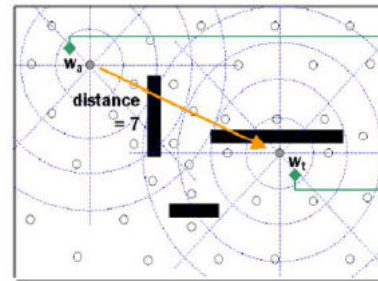
廈門大學
XIAMEN UNIVERSITY

Distribution Map



Left 4 Dead

Waypoints with LOS



Killzone

Territory Representation



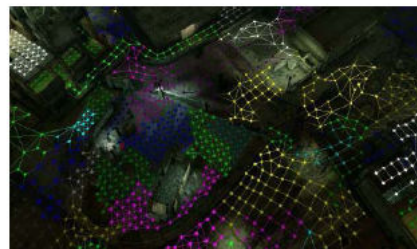
Halo2

Tactical Point System



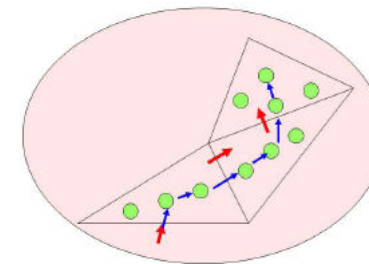
Halo

Tactical Map Clustering



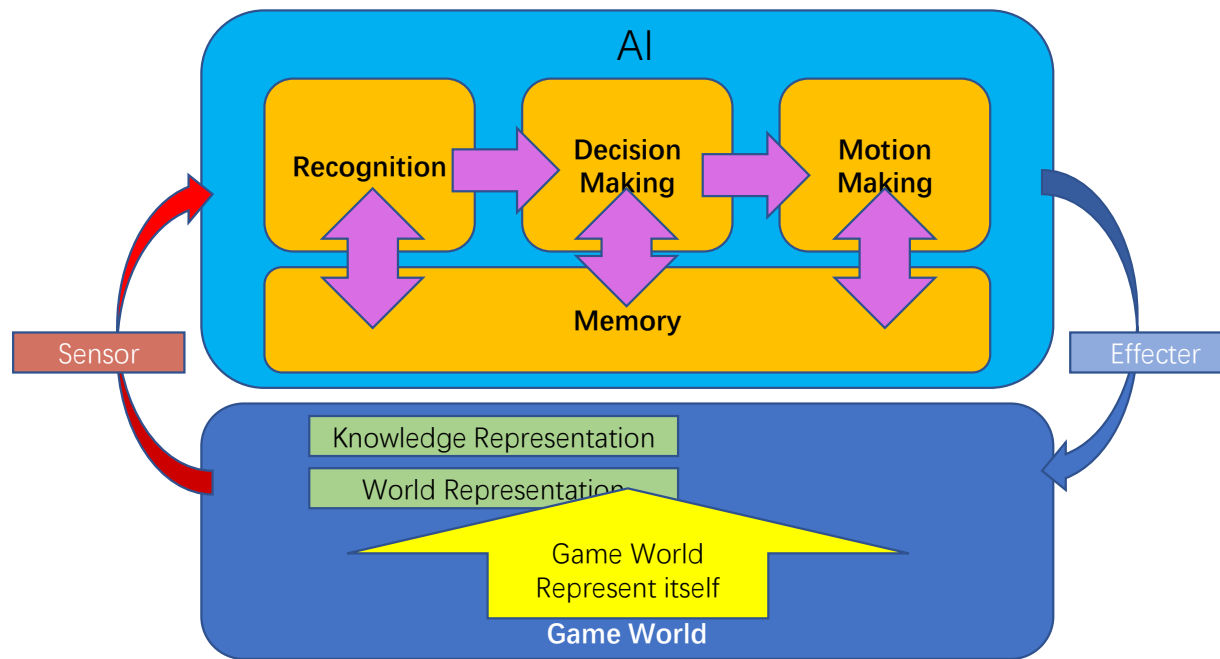
Killzone2

NavMesh-Waypoint
Hierarchical Representation

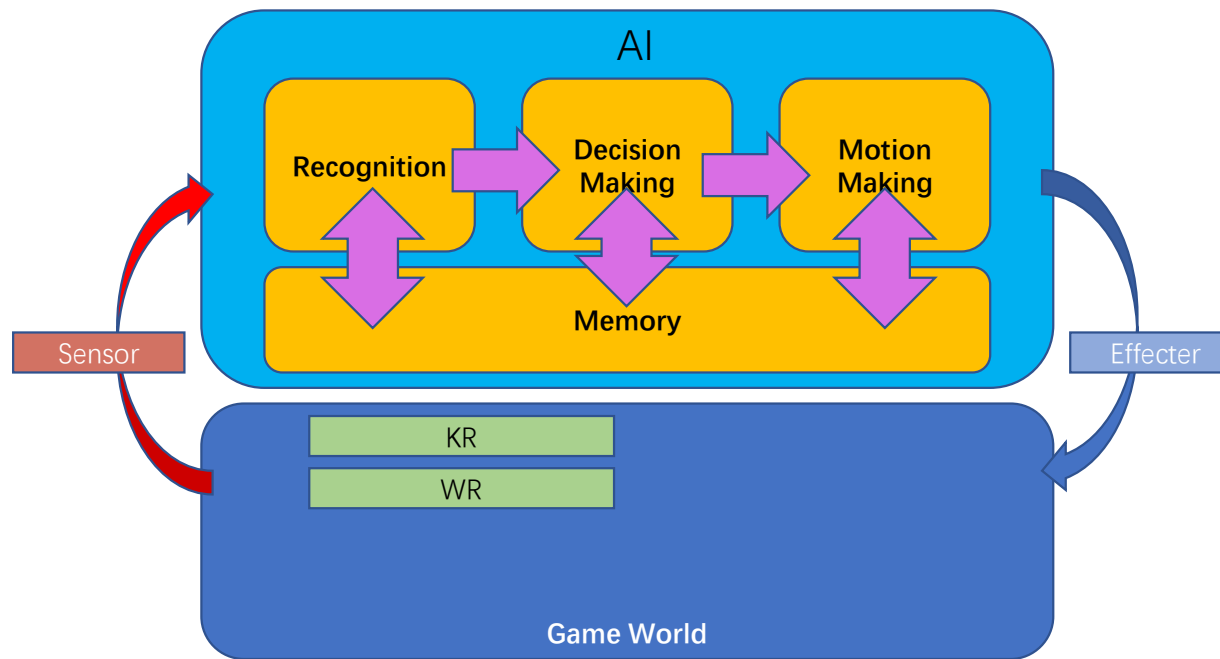


Assassin's Creed

Agent Architecture

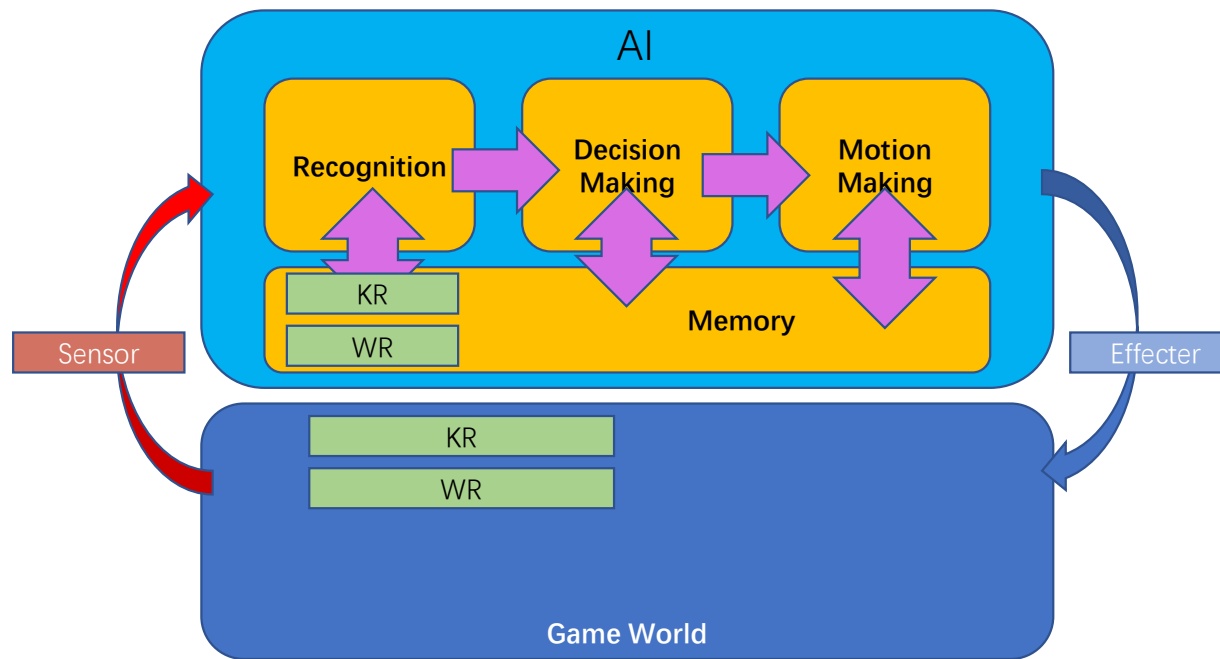


Agent Architecture



KR and WR enforce and expand sensors.

Agent Architecture



Type of KR and WR = Type of Knowledge in Memory (via sensors)

Affordance Representation

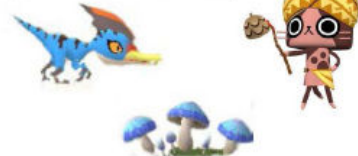
Affordance Representation
= Actions enable in the environment



FROM SOFTWARE

提案の流れ

ランボス・アフォーダンス
Attackable



キノコ・アフォーダンス
Eatable

提案テーブル		優先度
	攻撃したい	60
	剥ぎ取りたい	80
	食べたい	80
	寝たい	20
	進みたい	60

・実行不可能な提案を伏せます

- ◆ Genre: Action
- ◆ Developer: FROM SOFTWARE
- ◆ Publisher: CAPCOM
- ◆ Hardware: PSP
- ◆ Year: 2011

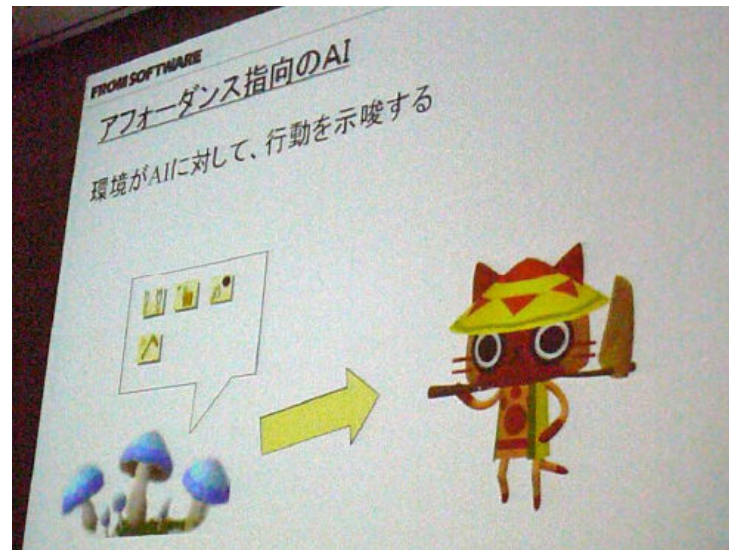
Each object has Affordance Representation.

Affordance Representation

Affordance Representation
= Actions enable in the environment

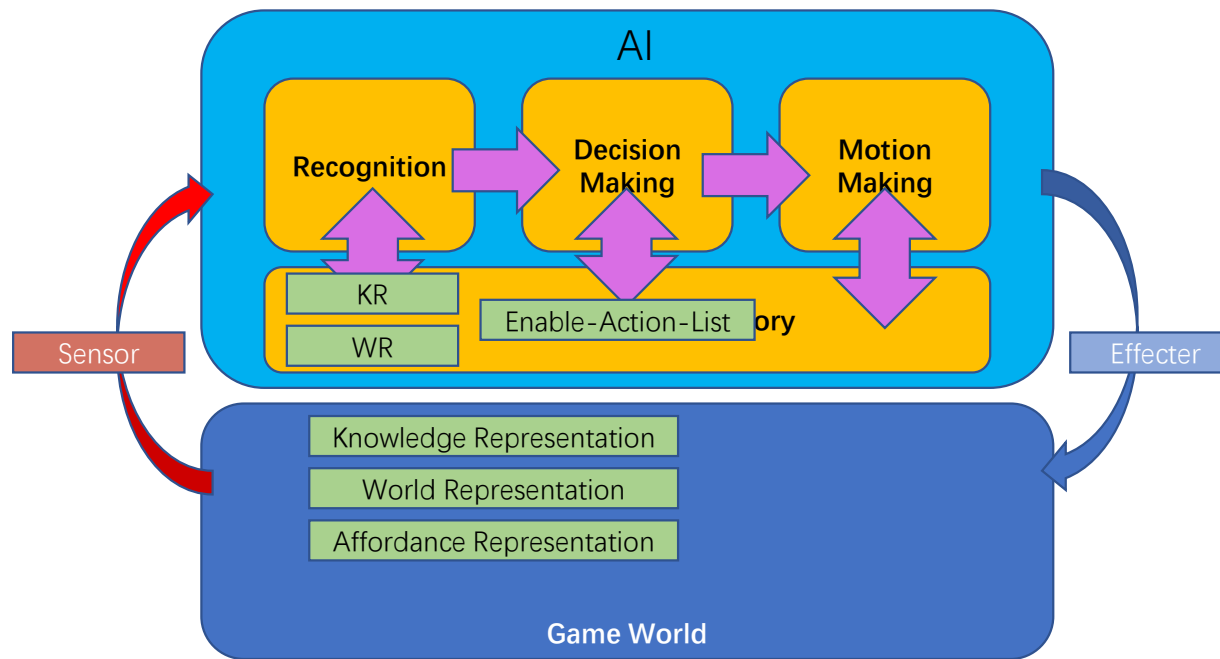


- ◆ Genre: Action
- ◆ Developer: FROM SOFTWARE
- ◆ Publisher: CAPCOM
- ◆ Hardware: PSP
- ◆ Year: 2011



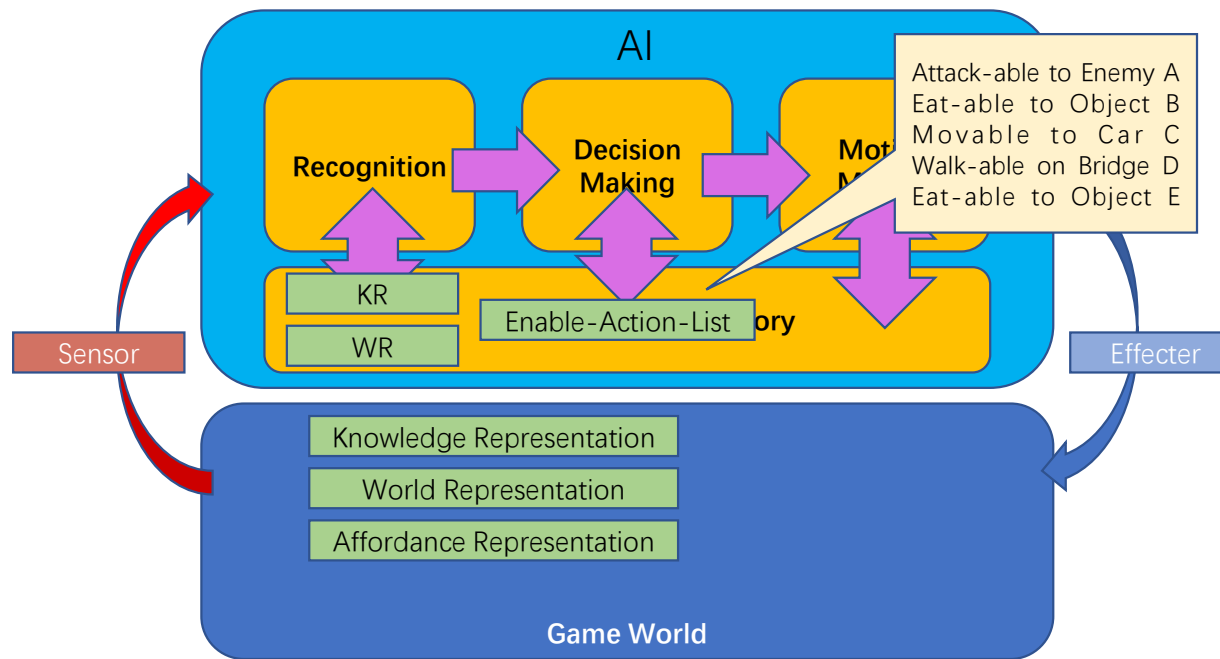
Each object has Affordance Representation.
= Each object suggests "Enabled-Action" to AI.

Agent Architecture



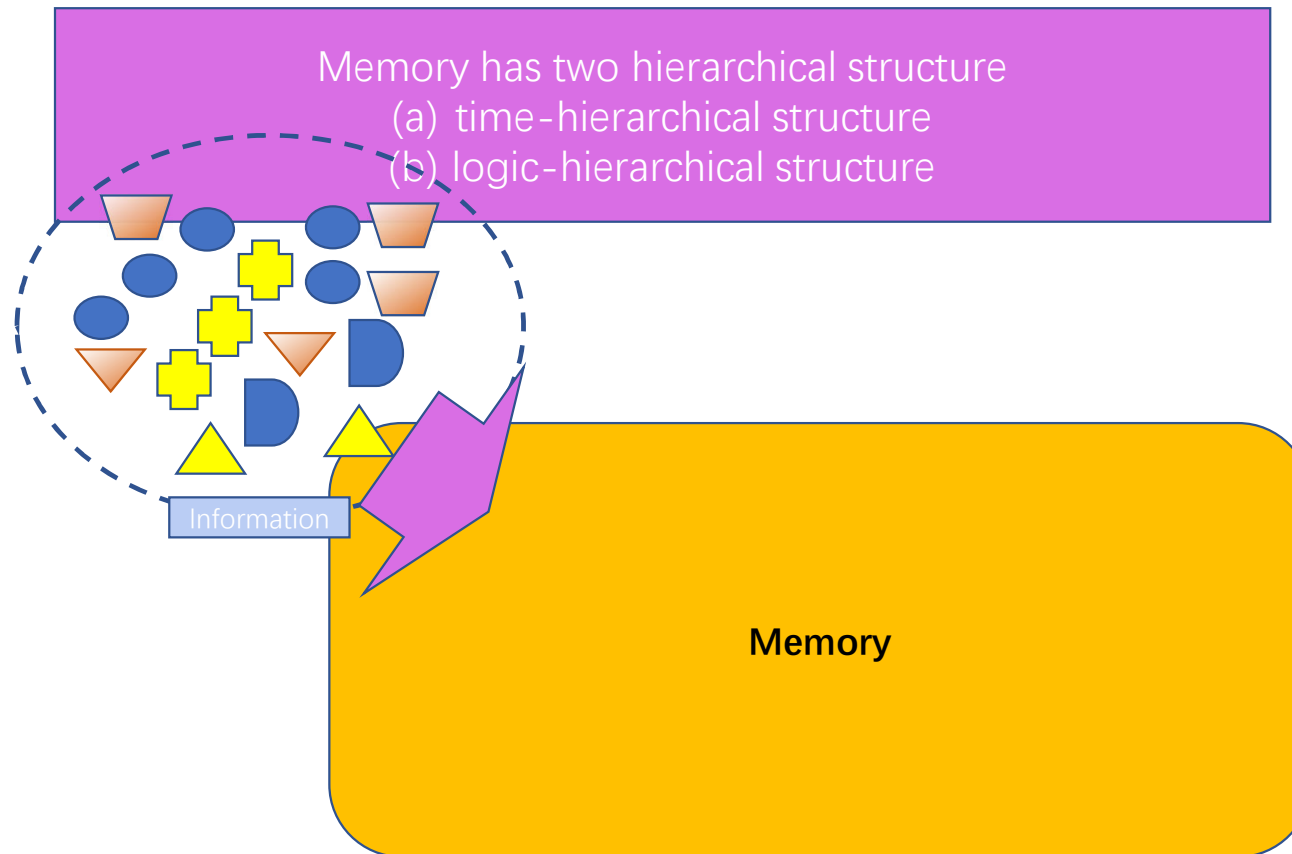
Affordance Representation = Afforded Action-List in the Environment (via sensor)

Agent Architecture



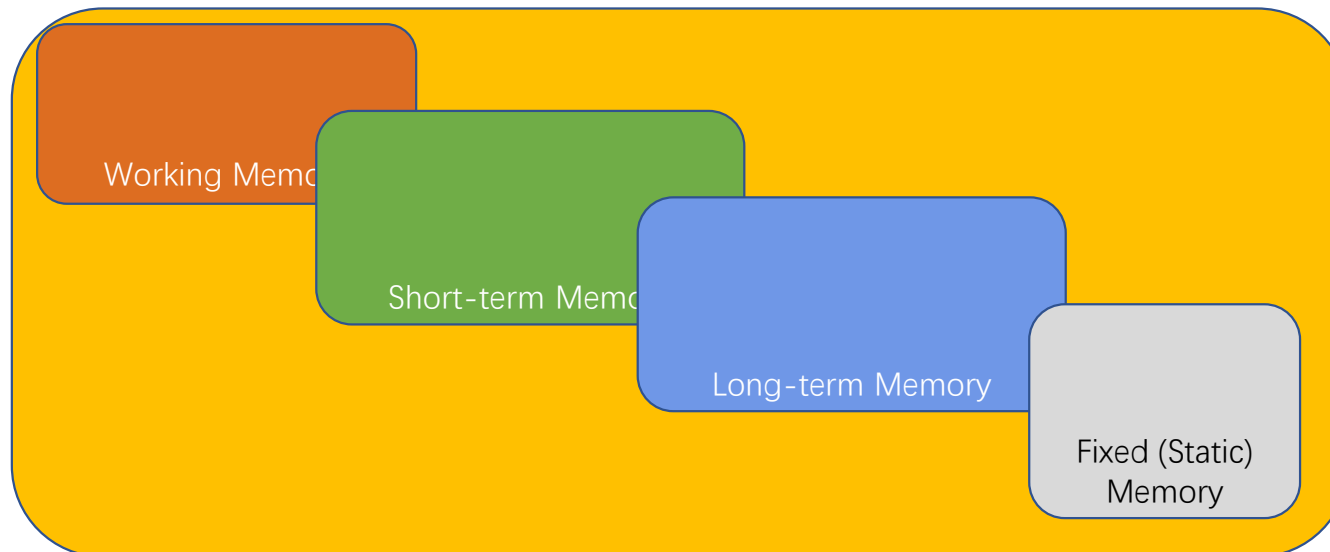
Affordance Representation = Afforded Action-List in the Environment (via sensor)

Memory Structure & Management

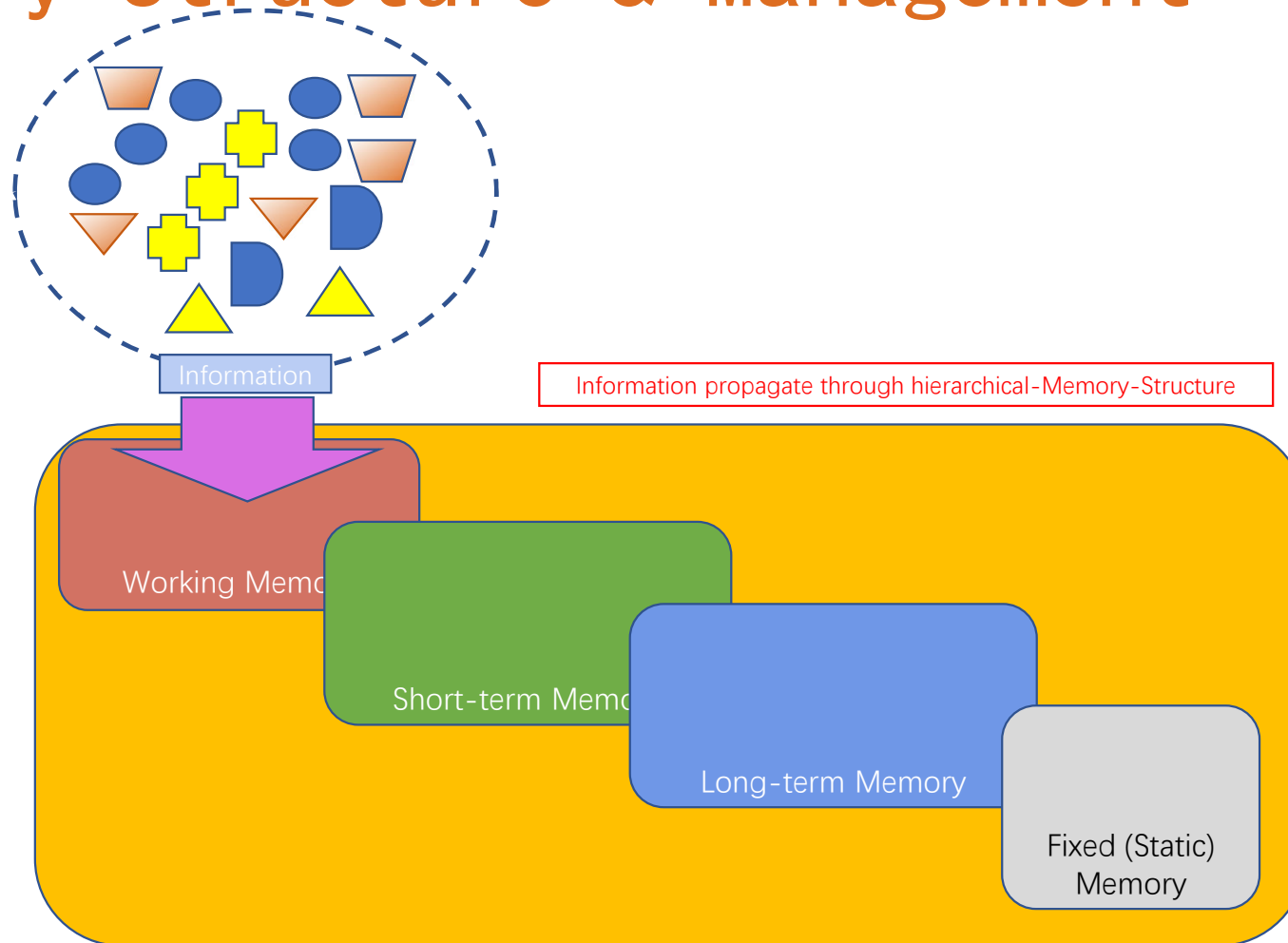


Memory Structure & Management

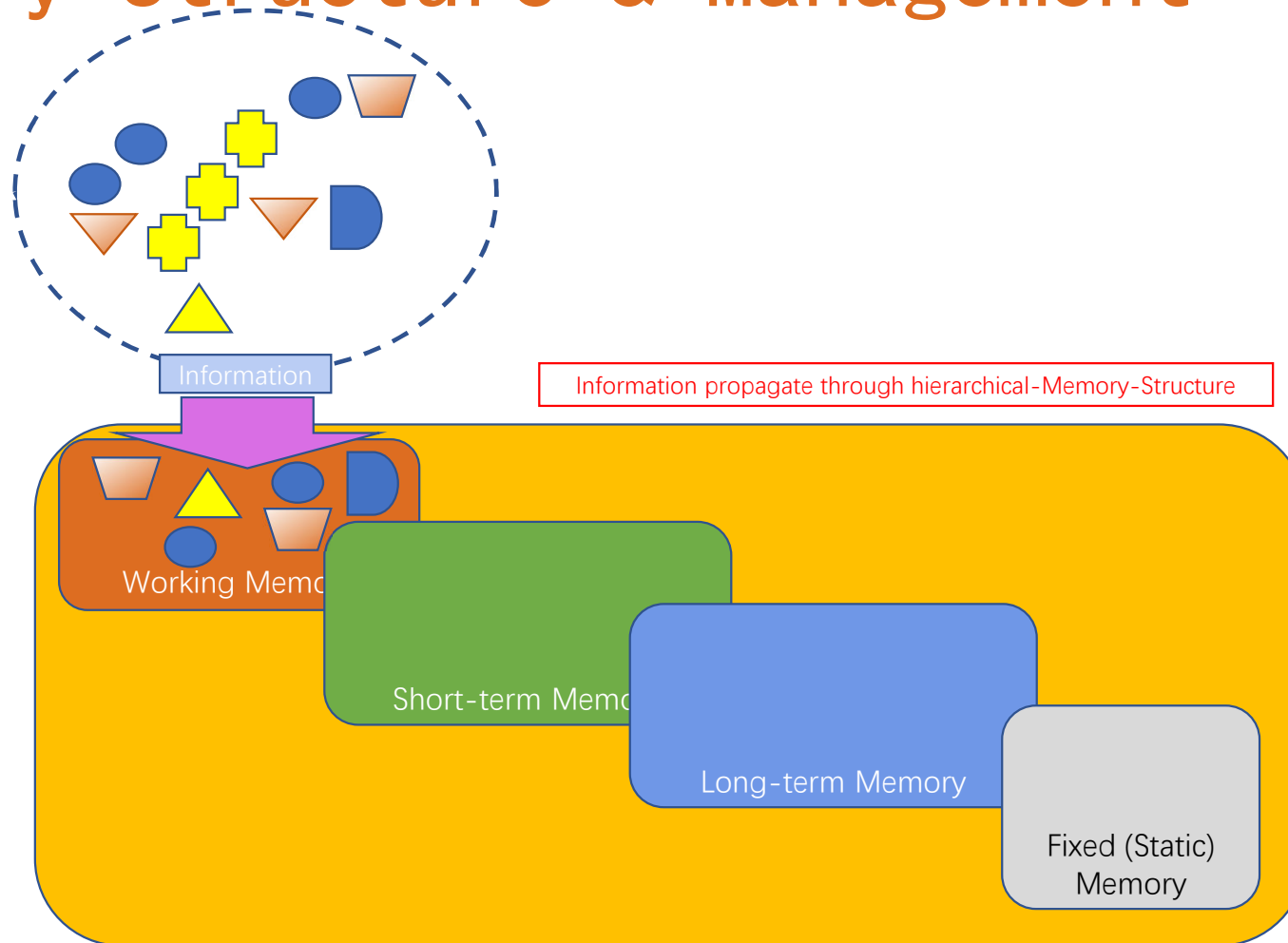
Memory has two hierarchical structure
(a) time-hierarchical structure
(b) logic-hierarchical structure



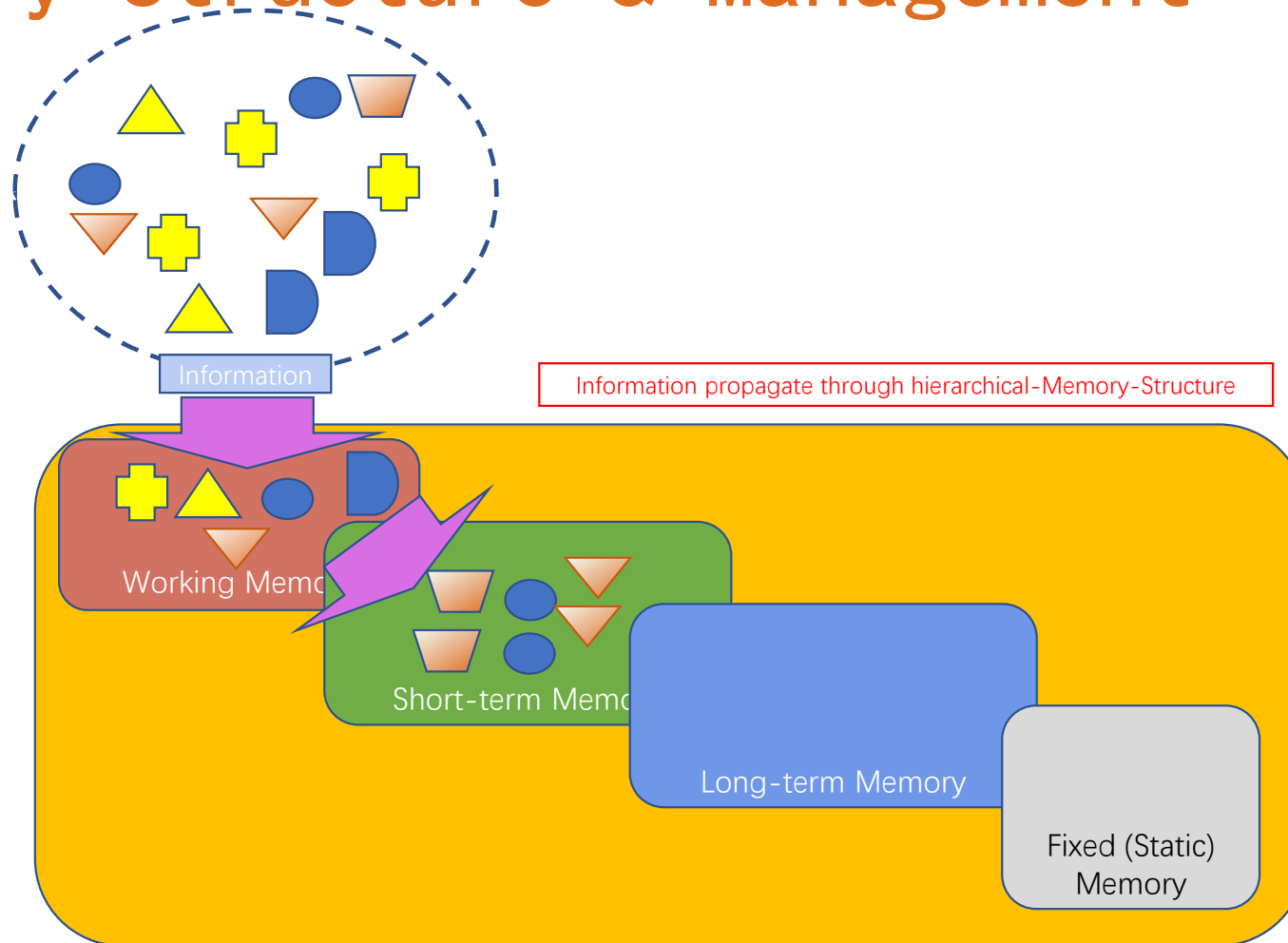
Memory Structure & Management



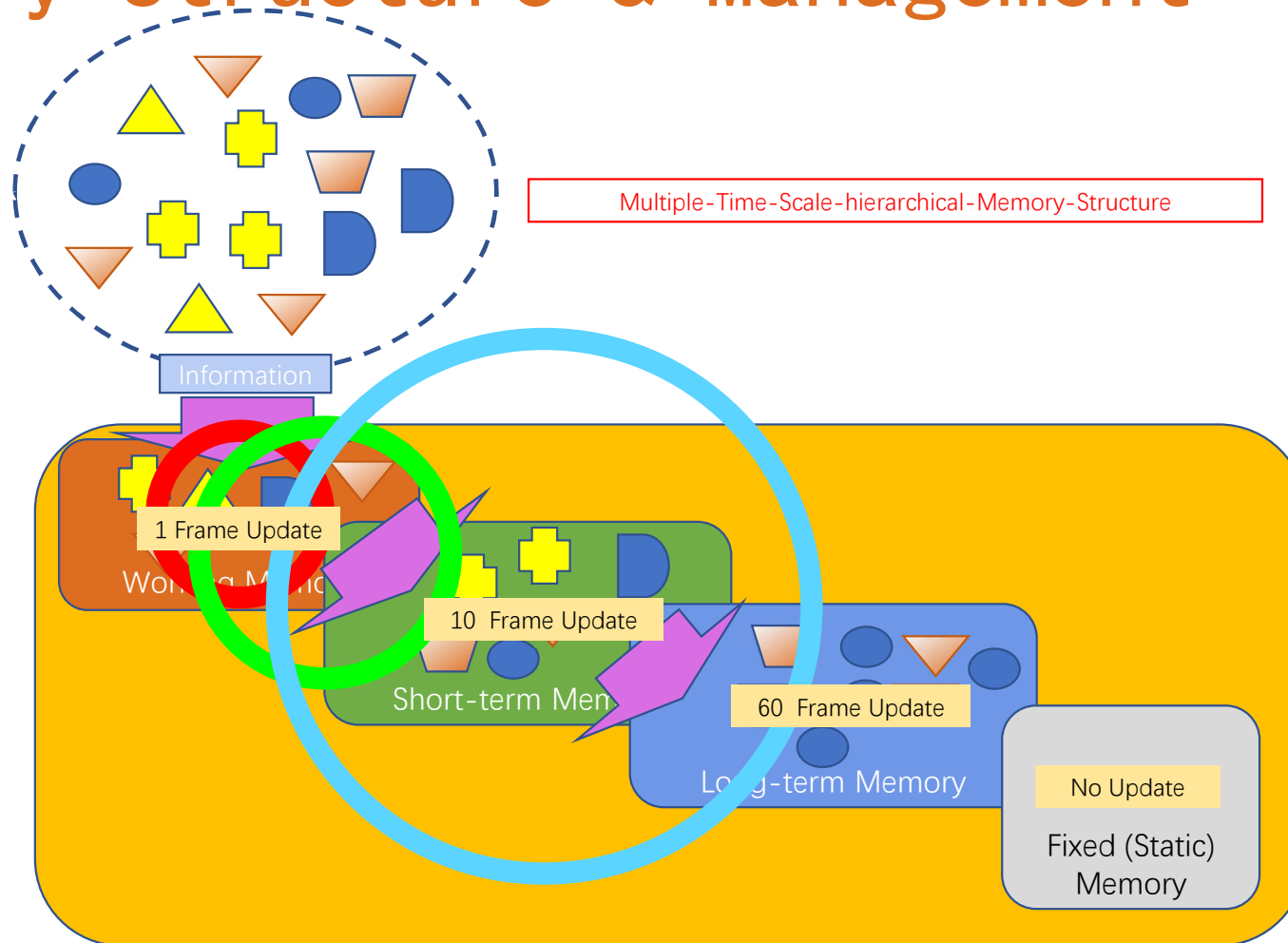
Memory Structure & Management



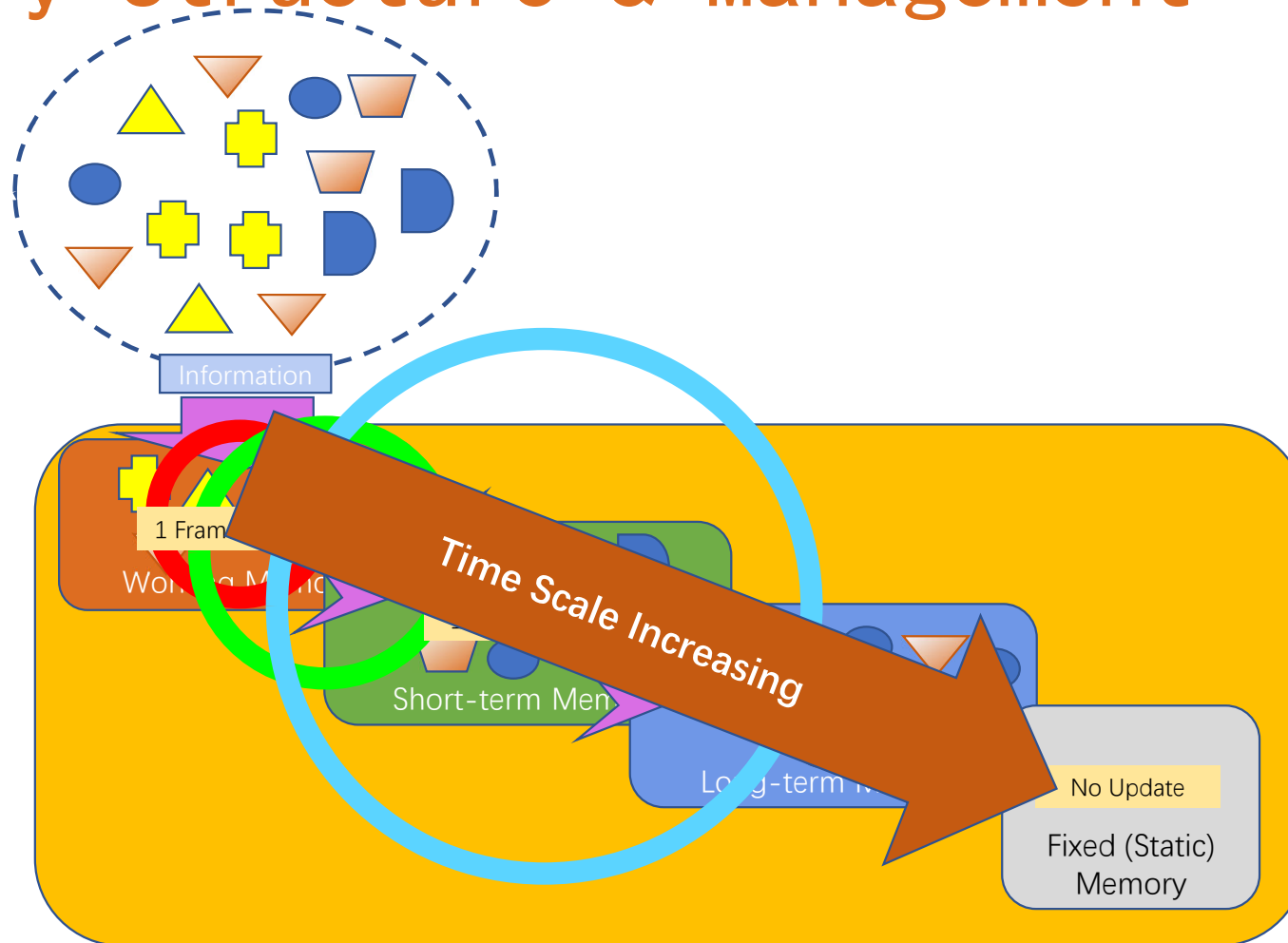
Memory Structure & Management

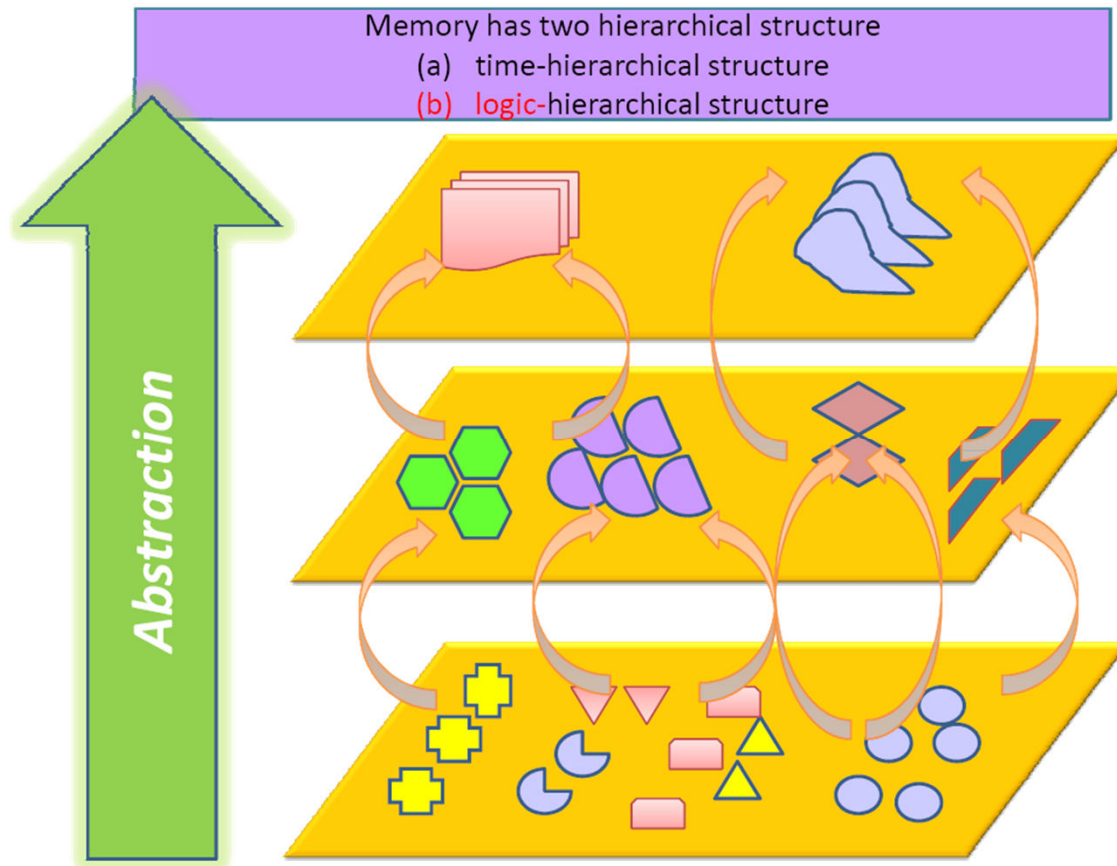


Memory Structure & Management

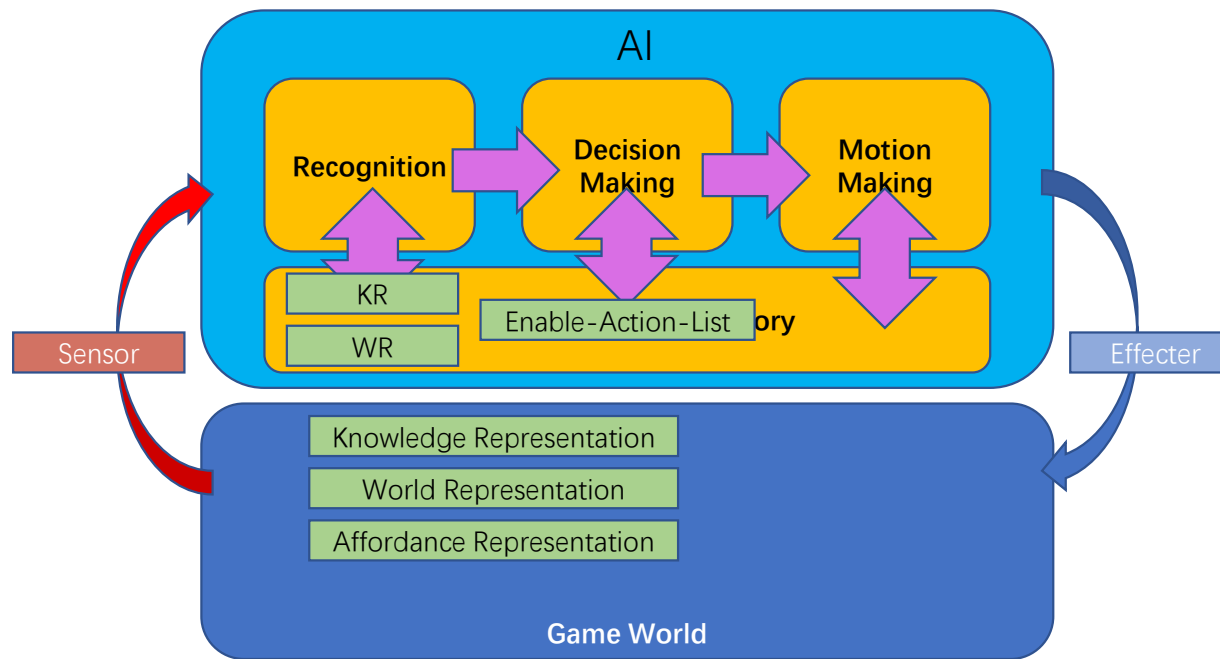


Memory Structure & Management



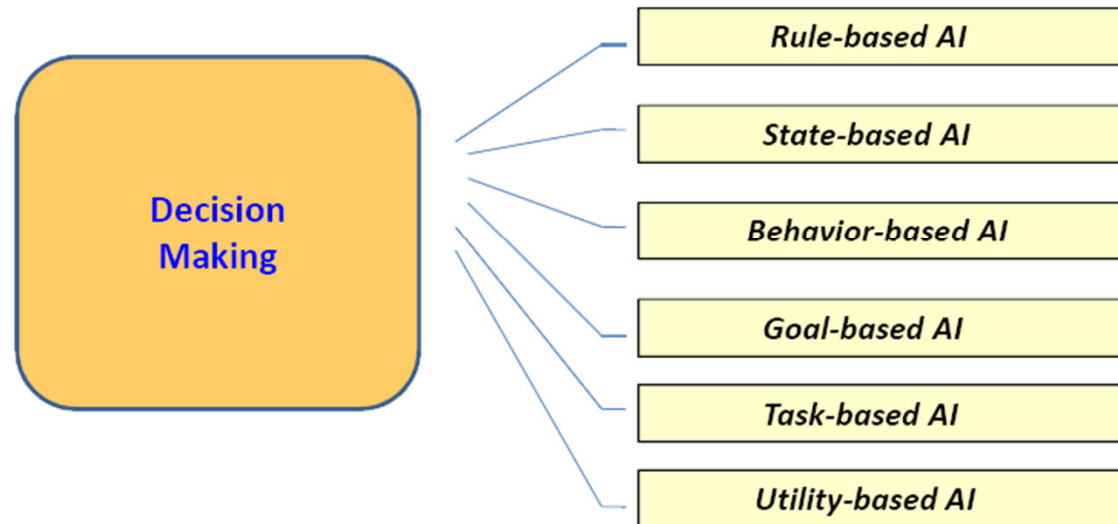


Decision Making



Decision Making

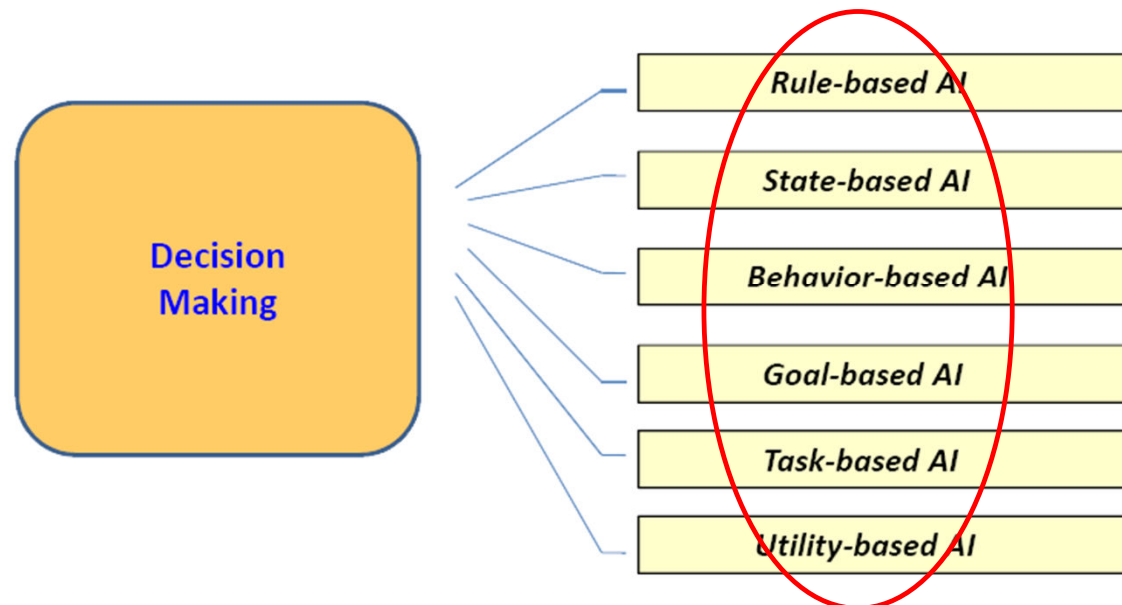
Decision Making Techniques are classified to **6** types



"Something- Based" means a Decision-Making-Method use **Something** as unified-form of thinking.

Decision Making

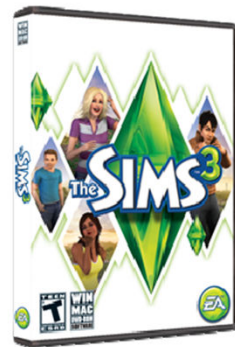
Decision Making Techniques are classified to **6** types



"Something- Based" means a Decision-Making-Method use **Something** as unified-form of thinking.

Decision Making

Rule-based AI



- ◆ Genre: Simulation
- ◆ Developer: Maxis
- ◆ Publisher: EA
- ◆ Hardware: PC, PS3
- ◆ Year: 2009

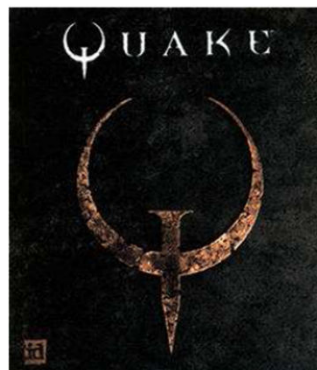
How Traits Affect Socializing

- ◆ Traits also affect how they **respond** to social interactions initiated by others
- ◆ How a Sim responds to a social depends on a series of production-rules:

```
TryingToBe.Funny -> Neutral
TryingToBe.Funny && Repetition -> Boring
TryingToBe.Funny && LTR < -20 -> Insulting
TryingToBe.Funny && Target.GoodSenseOfHumor -> Funny
```

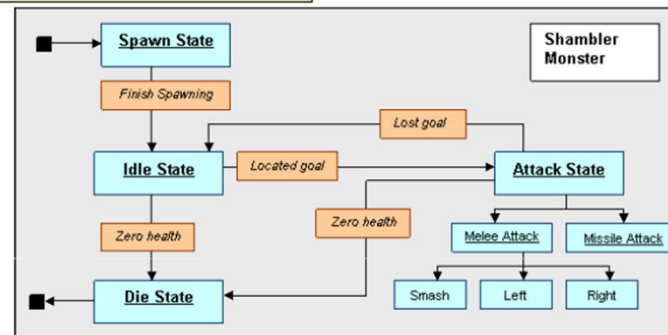


Decision Making



- ◆ Genre: SciFi-FPS
- ◆ Developer: id Software
- ◆ Publisher: GT Interactive
- ◆ Hardware: PC
- ◆ Year: 1996

State-based AI



Decision Making

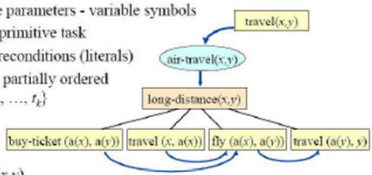
Task-based AI

Methods (Continued)

- Partially ordered method: a 4-tuple
 $m = (\text{name}(m), \text{task}(m), \text{precond}(m), \text{subtasks}(m))$
- $\text{name}(m)$: an expression of the form $n(x_1, \dots, x_n)$
» $x_1, \dots, x_n$ are parameters - variable symbols
- $\text{task}(m)$: a nonprimitive task
- $\text{precond}(m)$: preconditions (literals)
- $\text{subtasks}(m)$: a partially ordered set of tasks $\{t_1, \dots, t_k\}$

air-travel(x,y)

task: travel(x,y)
precond: long-distance(x,y)
network: $u_1 = \text{buy-ticket}(a(x), a(y))$, $u_2 = \text{travel}(x, a(x))$, $u_3 = \text{fly}(a(x), a(y))$
 $u_4 = \text{travel}(a(y), y)$, $\{(u_1, u_2), (u_2, u_3), (u_3, u_4)\}$



Copyright: Lecture slides for Automated Planning
Licensed under the Creative Commons Attribution-NonCommercial-ShareAlike License: <http://creativecommons.org/licenses/by-nc-sa/2.0/>

Individual AI : Application

HTN PLAN (non-interruptible) - [BOT] Tremethick

DECOMPOSITION

TASK LIST

```
(!forget active_plan **)  
(!remember - active_plan medic_revive [Soldier:TimmermanV])  
(!log_color magenta "Medic reviving nearby entity.")  
(!broadcast friendlies 30.0 10.0 medic_reviving [Soldier:TimmermanV])  
(!select_target [Soldier:TimmermanV])  
(!walk_segment (2370 2369 2368 2367 2366 2365 ... 5416) standing auto () () )  
A (!select_weapon wp_online_mp_bot_revive_gun)  
(!use_item_on_entity [Soldier:TimmermanV] crouching)  
(!forget active_plan **)
```

ACTIVE TASK INFO

AIHTNPrimitiveTaskSelectWeapon -

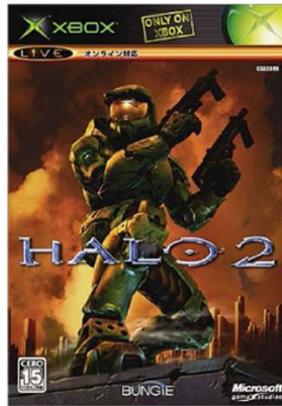
HTN (Hierarchical Task Network)



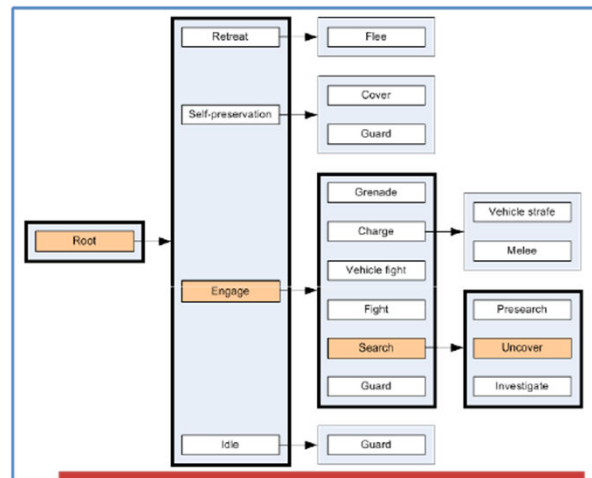
- Genre: FPS
- Developer: Guerrilla Games
- Publisher: SCE
- Hardware: PlayStation 3
- Year: 2009

Decision Making

Behavior-based AI



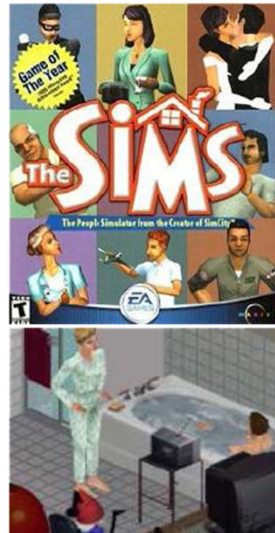
- ◆ Genre: SciFi-FPS
- ◆ Developer: BUNGIE Studio
- ◆ Publisher: Microsoft
- ◆ Hardware: Xbox, Windows, Mac
- ◆ Year: 2004



Behavior-Tree (has become very popular method .)

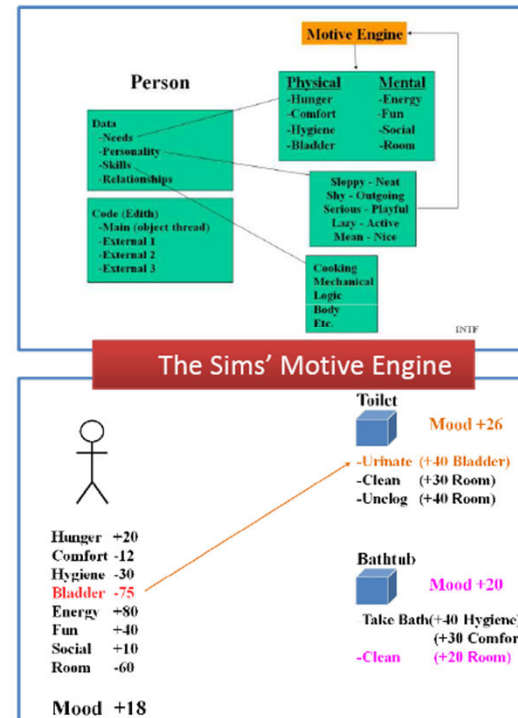
- (1) Hierarchical Structure of Box
- (2) Each box has some behaviors
- (3) Each box has one selection-rule
(random, sequential, priority-order...)
- (4) Finally, one behavior is selected at end-of-tree.

Decision Making

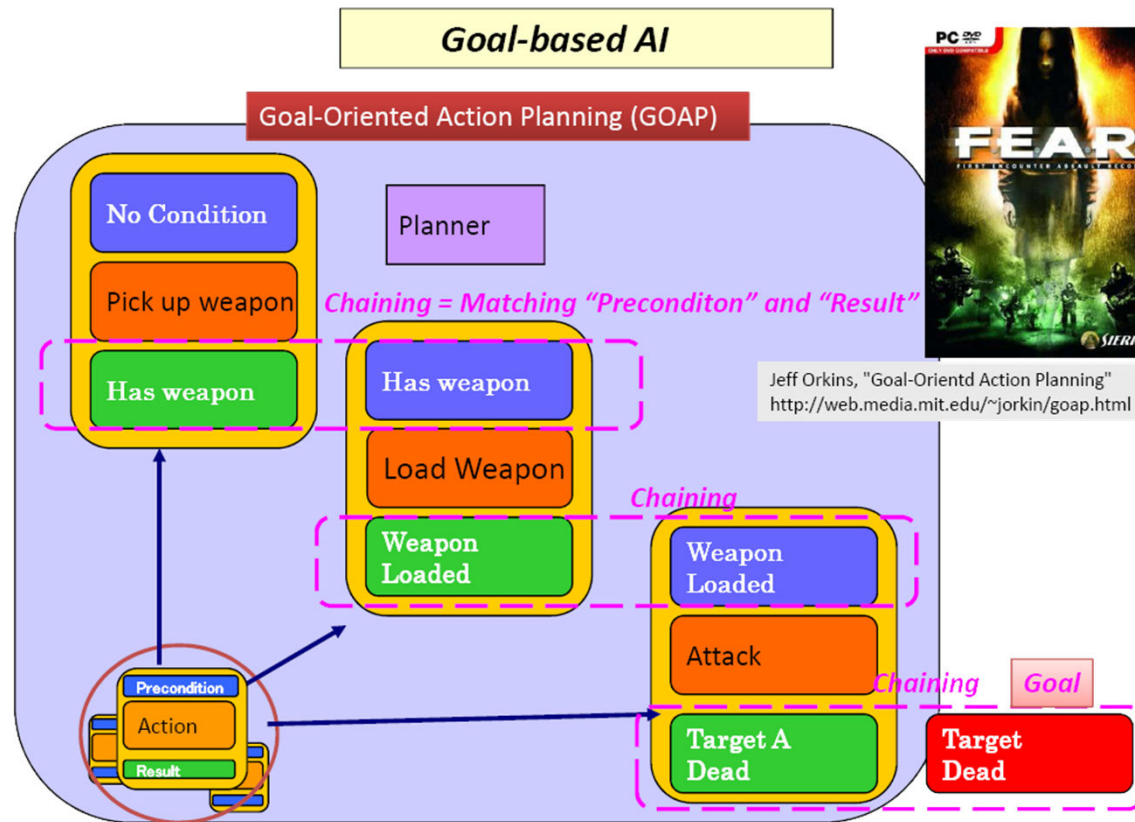


- ◆ Genre: Simulation
- ◆ Developer: Maxis
- ◆ Publisher: EA
- ◆ Hardware: PC
- ◆ Year: 2000

Utility-based AI



Decision Making



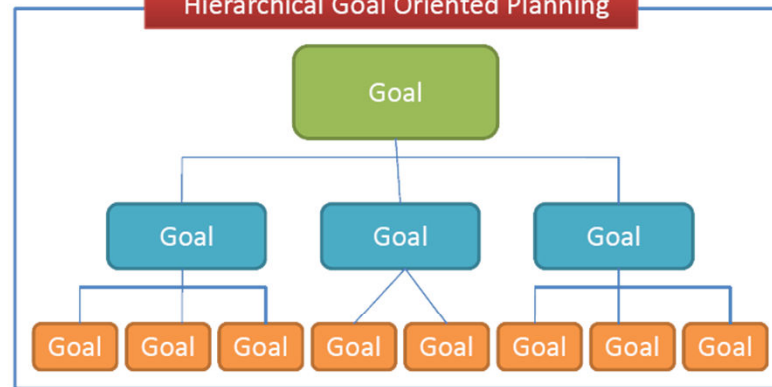
Decision Making

Goal-based AI

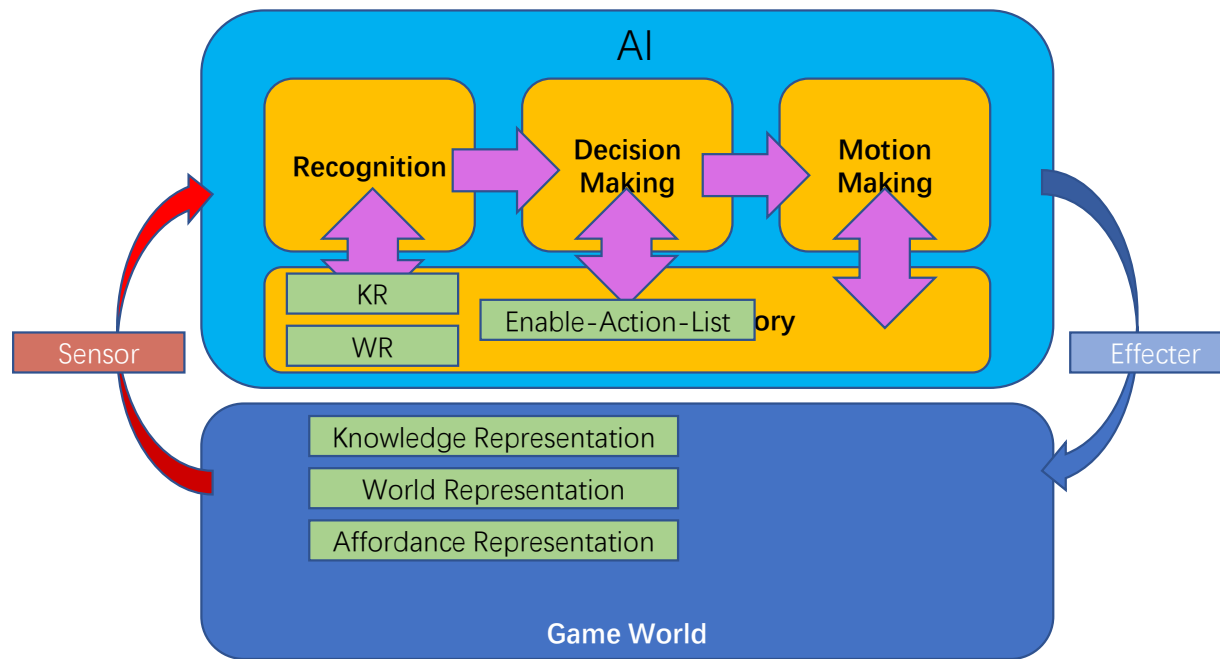
Hierarchical Goal Oriented Planning



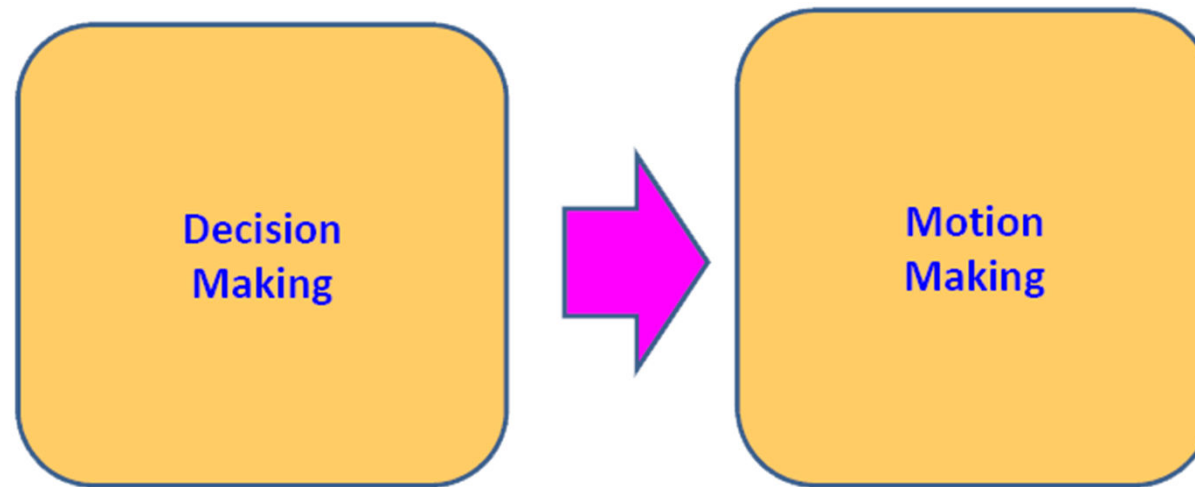
- ◆ Genre: Robot Action
- ◆ Developer: FROM SOFTWARE
- ◆ Publisher : SEGA
- ◆ Hardware: XBox360
- ◆ Year: 2006



Motion Making

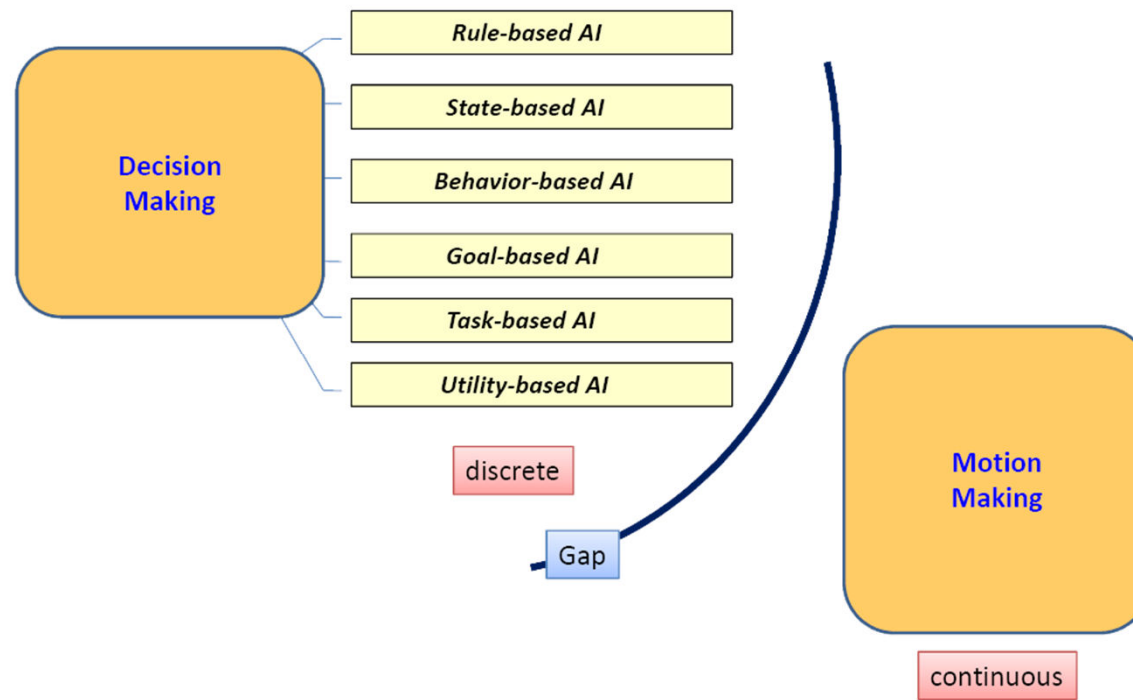


Motion Making

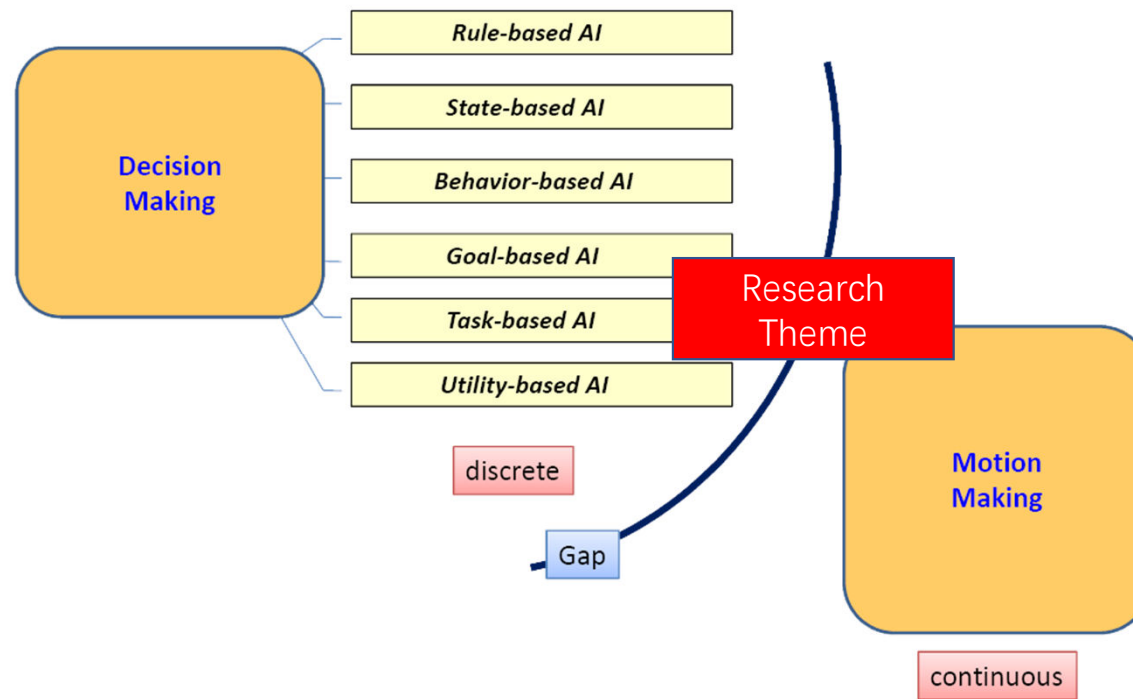


In this field, there are many, many models between DM and Motion Making.

Motion Making



Motion Making



Research Themes



- Architecture
- Knowledge/World/Affordance Representation
- Memory Structure & Memory management
- Sensor
- Decision Making
- ...and Synthesis of all these things to make
- “Real” intelligence.



廈門大學
XIAMEN UNIVERSITY

2.3 Procedural Techniques

--自强不息，止于至善--

NO MAN'S SKY

EVERY ROCK PROCEDURAL

EVERY ROCK PROCEDURAL

GAMESPOT

- Stolen terms (bits, space, scenarios): Procedural Content Generation for Games: A Survey
 - https://course.ccs.neu.edu/cs5150f13/readings/hendrikx_pcgg.pdf
- Search-Based Procedural Content Generation: A Taxonomy and Survey
 - https://course.ccs.neu.edu/cs5150f13/readings/togelius_sbpcg.pdf
- PCG in Games: A textbook and an overview of current research (2016)
 - <http://pcgbook.com>
- <http://pcg.wikidot.com/>

Free on website

About the book

Welcome to the Procedural Content Generation in Games book. This is, as far as we know, the first textbook about procedural content generation in games, aka PCG. As far as we know it is also the first book-length overview of the research field. We hope you find it useful, whether you are studying in a course, on your own, or are a researcher.

Content is king!

PROCEDURAL CONTENT GENERATION

Procedural Content Generation



- Use of computation instead of manual effort to produce elements of gameplay. Why?
 - Design aspects of the game
 - Save development cost (?)
 - Save storage or main memory (“infinite games”)
 - Adapt aspects of the game (player models)
- In what games have you seen this? What’d you think?
- What sorts of problems with PCG? What’s well suited?

What is PCG



- Generating game content (levels, narrative, quests, visuals, sound, etc.) algorithmically.
 - Game **bits** (characters, visuals, sound)
 - Game **space** (levels, worlds, puzzles)
 - Game **scenarios** (story, quests, history)

Game Bits (characters, visuals, sound)

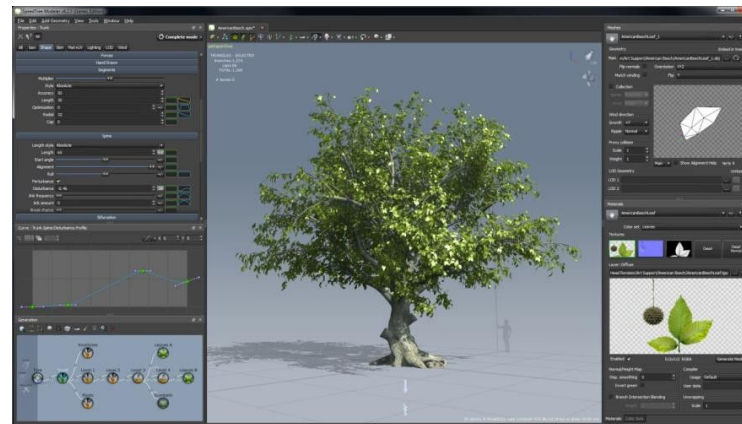


Borderlands series



No Man's Sky

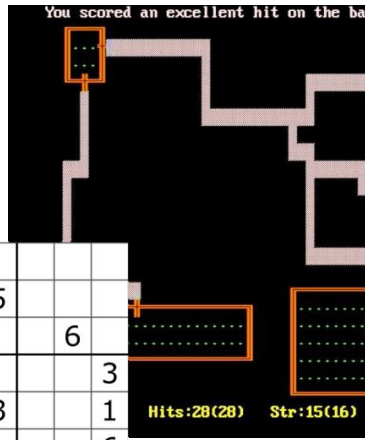
SpeedTree



Game Space (levels, worlds, puzzles)

Sudoku

5	3		7				
6			1	9	5		
	9	8				6	
8			6				3
4			8		3		1
7			2				6
	6				2	8	
			4	1	9		5
			8			7	9



Rogue (1980)



Diablo 3



Bloodborne, Chalice Dungeons



Spelunky

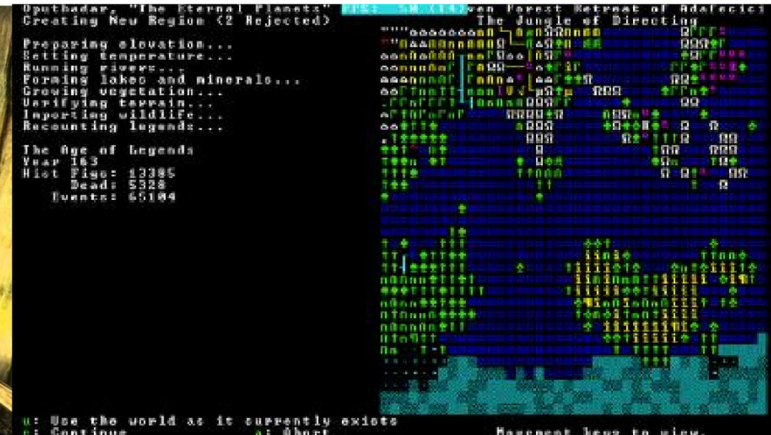


Minecraft

Game Scenarios (story, quests, history)



Skyrim, radiant quests



Dwarf Fortress



Diablo 2, randomizing situations



Shadow of Mordor



MASS EFFECT™

ANDROMEDA

kotaku.com/the-story-behind-mass-effect-andromedas-troubled-five-1795886428

Why use PCG?



- Reducing cost of creating content?
 - Yes, when clever (SpeedTree), but can add complications (Mass Effect Andromeda)
 - Exponentially increasing costs (levels, maps, missions, etc.)
- CONTENT IS KING
 - If its interesting/personal/exciting to the player (Borderlands 2 guns), not if its bland oatmeal (No Man's Sky)
 - Computed content saves:
 - development costs
 - Storage and/or main memory
- Keep player playing/replaying
 - Adapting to player (Left4Dead AI director), new every time (Spelunky), or subarea (Bloodborne Chalice Dungeons)

Two Major Ways to use PCG



- Runtime PCG: customization, dynamic adjustment, replayability
 - Generate content live in game (No Man's Sky, Spelunky, Minecraft, Horizon Zero Dawn, etc)
- Offline/Design-time/Developer PCG: Speed up design of static content
 - generate, pick best, and save content
 - generate then edit
 - generate and incorporate (Speedtree)

PCG high-level Methods



- Rule systems
- Generative Grammars
- Search
- Constraint Solving

PCG high-level Methods



- **Rule Systems** allow for emergent output from simple rules. But hard to control output.
- **Generative grammars** can create high-quality output, but depend upon expert authoring of components and rules
- **Search** cuts back on authoring burden to just a heuristic and searchable representation, but these are unintuitive for many designers

How'd they do it?

How'd they do it?

译者:tkchu

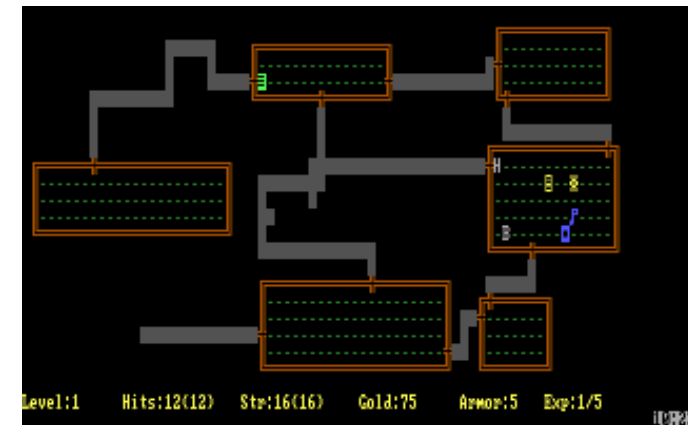
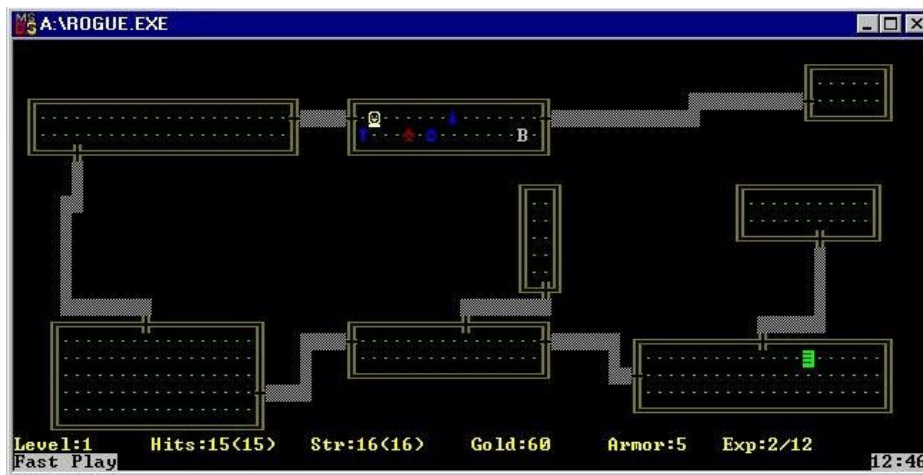
History



- Level generation was an early feature of games
- Games weren't professionally developed
- Memory and storage was limited
- Processors also limited
- Tradeoff between memory/storage and processor was important
 - Random generation was the answer

History

- Rogue (AI Design, 1983)
 - <http://www.abandonia.com/en/games/982/Rogue+-+The+Adventure+Game.html>
 - Granddaddy of all rpg dungeon-crawlers
 - Unlimited replay: dungeons + monsters PCG



History

- Elite (Acornsoft, 1984)
 - <http://www.iancgbell.clara.net/elite/>
 - Space trading (piracy, trade, military, etc.)
 - Original plan: 248 galaxies (~282 trillion), 256 solar systems each
 - Final: 8 galaxies
 - Planet pos., commodity prices, name/local details
 - Model for Wing Commander, Grand Theft Auto, EVE online; influenced Second Life, World Of Warcraft...



History



- The Sentinel (Firebird, 1986)
 - http://www.youtube.com/watch?v=9V_pgo3vgil
 - “The first VR game”
 - 10,000 levels
 - Pseudorandom number generators (again)
 - landscape from the 8-digit number given at the completion of a previous landscape
 - Not all landscapes tested; possible to skip difficult levels by rebeating earlier one
- Consolidated list:
 - http://en.wikipedia.org/wiki/Procedural_generation#Software_examples

Why industry considers PCG

- Development cost savings
- Replayability (e.g. easing permadeath)
- Customization/adaptation
- Dynamic difficulty adjustment
 - Flow theory
 - C.f. [http://en.wikipedia.org/wiki/Flow_\(psychology\)](http://en.wikipedia.org/wiki/Flow_(psychology))
 - <http://jenovachen.com/flowingames/designfig.htm>
 - [http://en.wikipedia.org/wiki/Flow_\(video_game\)](http://en.wikipedia.org/wiki/Flow_(video_game))

Design time PCG



- Graphical asset construction is also consuming development time
 - # of unique objects in the world
 - Players expect non-repetitive
 - Game dev times now 100s of man-years with huge design teams
- Cost savings therefore a strong motivation
- But there are risks:
 - Procedures can do stupid things that destroy the player experience
 - Game design process exists to maximize control of design

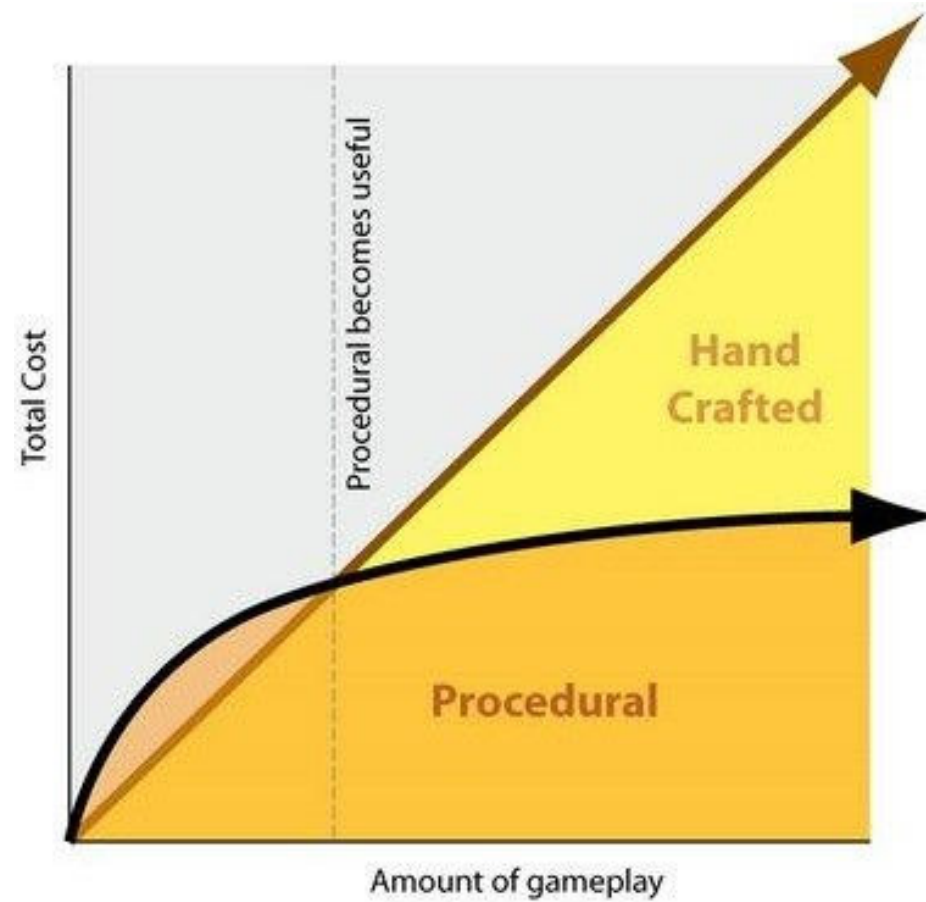
Design time PCG

- Typically a mix of designer provided assets and procedural use of assets
 - Speedtree
 - CityEngine:
 - Terragen
 - Star Trek Nemesis
 - Golden Compass
 - TV Commercials

CityEngine: <http://www.esri.com/software/cityengine/>



Economies of scale



自强不息，止于至善

Types of content

- Throw away
- Reusable
- Handcrafted anything is expensive and hard to modify
- The more handcrafted throwaway content, the less agile development, and more time spent on things the players don't appreciate
- PCG doesn't have to be human-level; just “good enough”

Run time PCG



- Players are different
- Preferences for pace and playstyle
 - Moderate challenge levels
 - Help avoid getting stuck
 - Adapt to player's tastes
 - Detecting player exploits
- When to use run-time PCG
 - When decisions can only be made at run-time
 - When you just can't pre-compute due to storage/memory limits
- Optimization problem
 - What is the set of content that delivers the optimal experience to the player given individual differences?
 - Example: rubber banding

Player Modeling



- What is optimal in PCG?
- Need a player model
 - Tells you something about the player
 - Makes predictions about player
- Challenges in player modeling:
 - What do you model?
 - Where do you get the model?
 - How do you use the model?

Promises



- Instancing in-game entities (especially appearance)
- NPC histories
- Puzzles that are harder or easier
- Puzzles/levels with more than one solution
- Story generation
- Pacing
- Different, multiple endings
- Meaningful choices
- Extending game lifetime

Pitfalls / Risks



- PCG can be NP-hard for anything non-trivial
 - But you can't always make players wait
- Thorough testing of run-time PCG is impossible
 - Best you can do is statistical sampling
- Offensive material
- Bad content
- Algorithm crash
- Meaningless activities
 - Easy to create quests, but if they don't connect to the larger game, no one cares

Hard problems in PCG



- Procedures structuring player's experience
 - Narrative
- Mental models of players
- Statistical player models
- Avoiding (dealing with) occasional catastrophic failure
- Algorithm safety
- Healing a broken level
- Social factors and full expressiveness with story/language generation
- “Deep understanding” vs. “generate & test”
- Personalized generated content
- Destiny
 - It's easy to have infinite levels, but dungeon crawling gets boring without a sense of progression/achievement
- **How do you know you are generating something interesting?**



廈門大學
XIAMEN UNIVERSITY

2.4 Reinforcement Learning

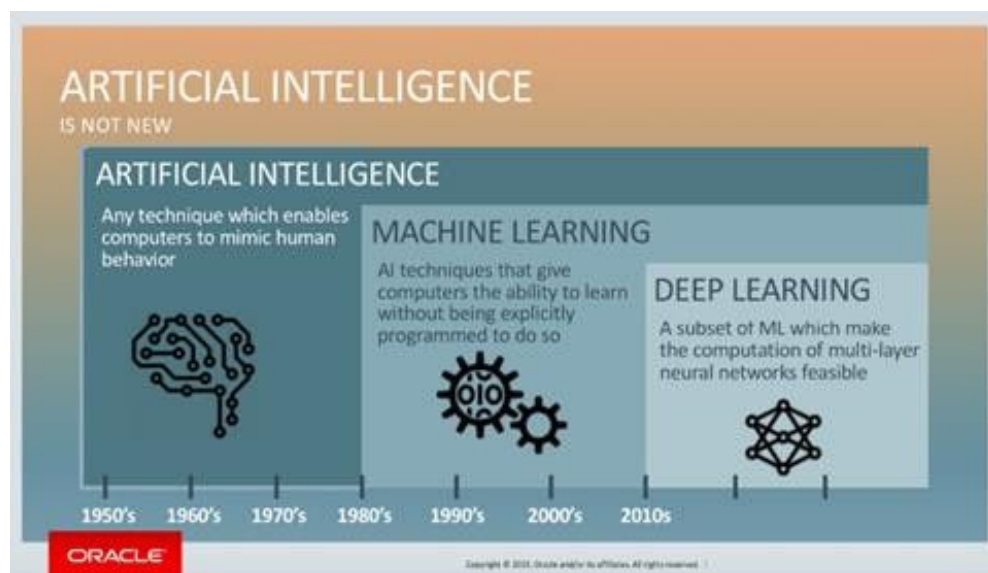
--自强不息，止于至善--

Situating RL

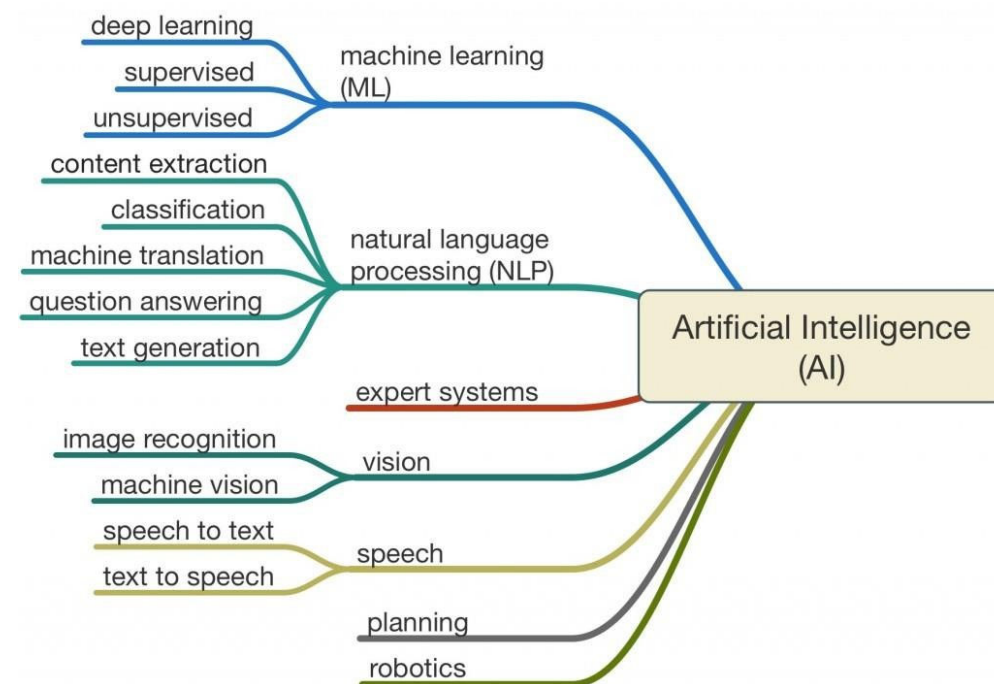


- The BLUF: RL is about learning from interaction about how to map situations to actions to maximize reward. Key elements:
 - Trial-and-error search: must discover which actions yield most reward by trying them
 - Delayed reward: actions may affect both immediate reward and also next situation and all subsequent rewards
- Different from:
 - Supervised learning: training set of labeled examples
 - Unsupervised learning: finding latent structure in unlabeled data
- RL: maximize reward signal rather than discover hidden structure

Organizing our thoughts

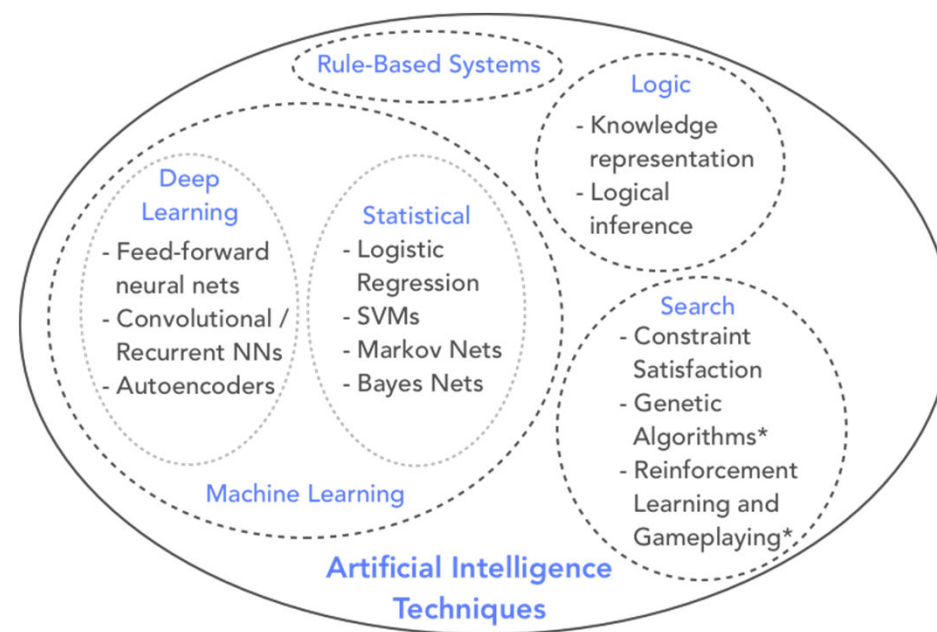
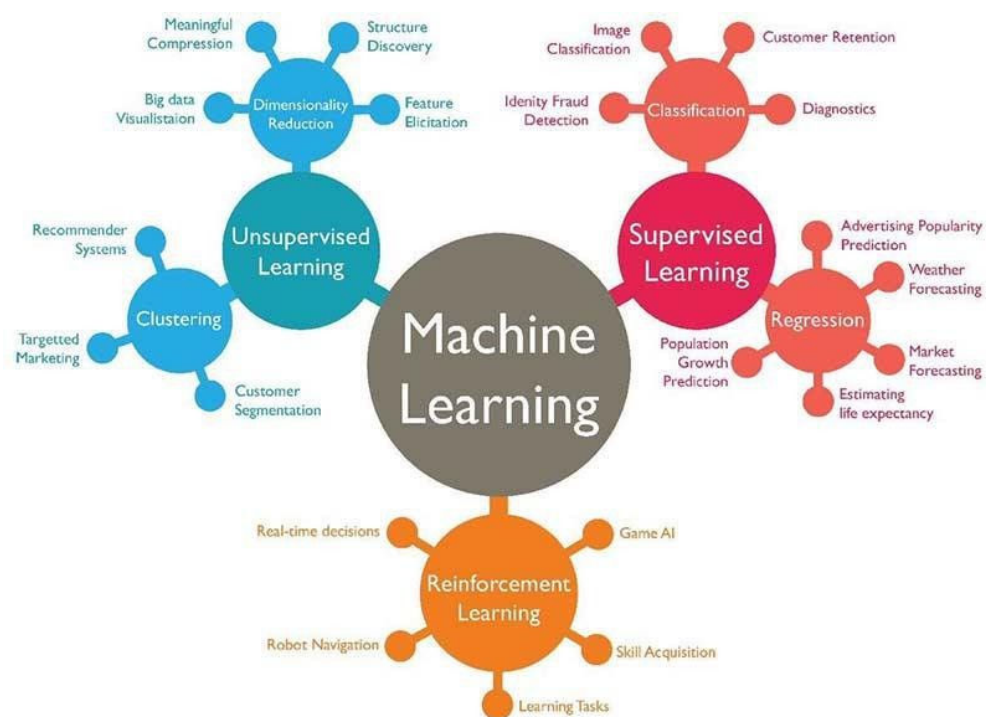


<https://blogs.oracle.com/bigdata/difference-ai-machine-learning-deep-learning>



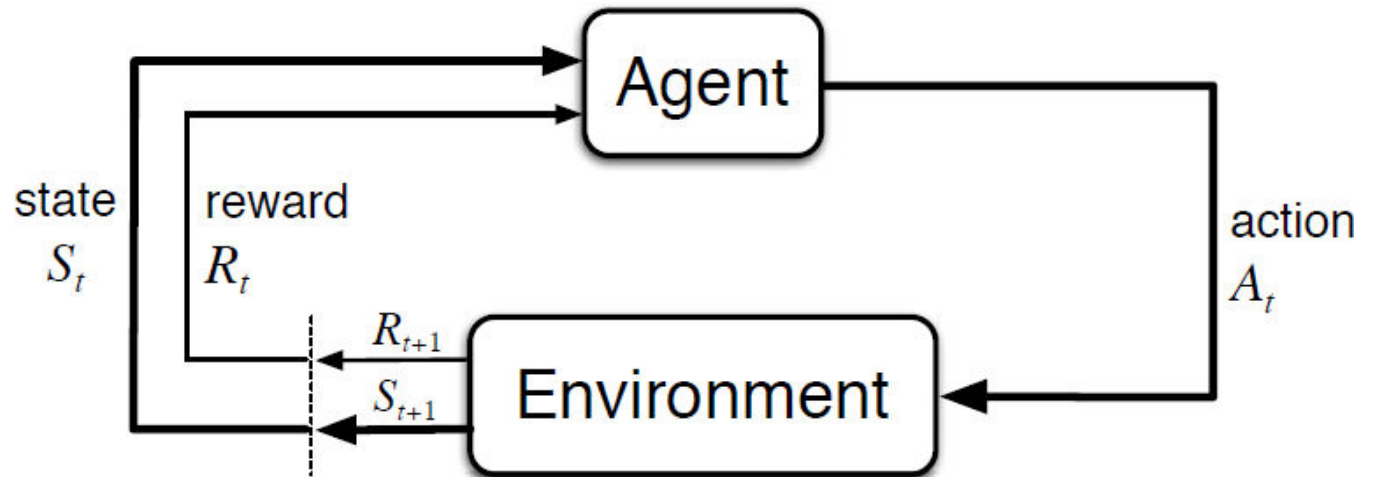
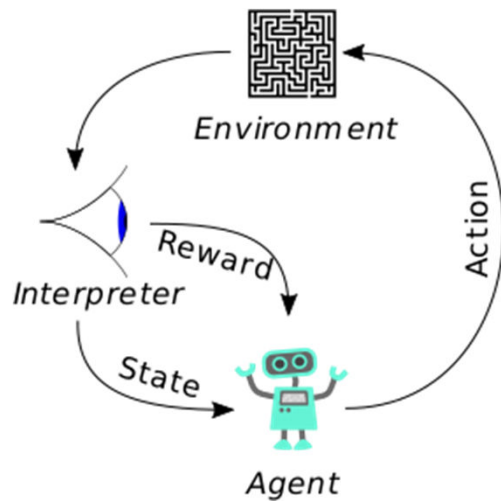
<http://www.legalexecutiveinstitute.com/artificial-intelligence-in-law-the-state-of-play-2016-part-1/>

Mind Map / Family Tree



<https://www.techleer.com/articles/203-machine-learning-algorithm-backbone-of-emerging-technologies/>

<https://blog.eloquent.ai/2018/08/15/the-ai-landscape/>



Sutton & Bartow, Fig 3.1

Fundamental Classes of Methods



- Dynamic programming
 - Mathematically well understood but require a complete and accurate model of the environment
- Monte Carlo methods
 - Model free and conceptually simple, but not well suited for step-by-step incremental computation
- Temporal-difference learning
 - Model free and fully incremental, but difficult to analyze

Also differ in efficiency/speed of convergence

RL Has Revolutionized AI Gameplay

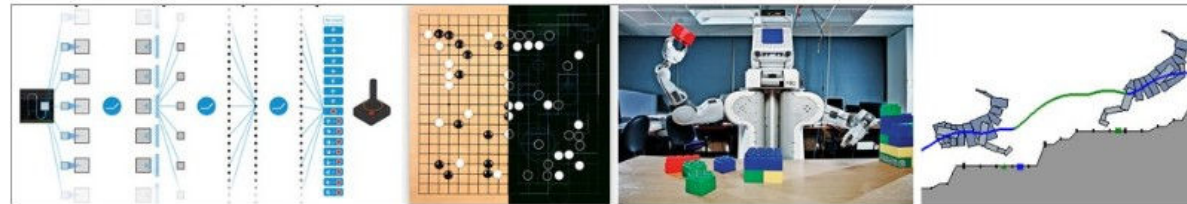


- Reinforcement Learning has been shown to be able to master many computer games
 - most Atari Games, Backgammon, Go, DOTA2, and StarCraft II.
- Reinforcement Learning (RL) is a class of algorithms that solve a Markov Decision Process. That is, the agent is given:
 - S: A set of all states the agent could encounter. Games: S is all configurations of pixels that the game engine can render
 - A: A set of all actions. Games: all player commands
 - T: A transition function that indicates the probability of transitioning from one state to another state if a particular action is chosen. We will assume that T is unknown.
 - R: A reward function that computes a score for every state in S. Games: reward given either based on in-game score, or for finishing a level, or achieving an objective.

Deep Reinforcement Learning: Pong from Pixels

May 31, 2016

This is a long overdue blog post on Reinforcement Learning (RL). RL is hot! You may have noticed that computers can now automatically [learn to play ATARI games](#) (from raw game pixels!), they are beating world champions at [Go](#), simulated quadrupeds are learning to [run and leap](#), and robots are learning how to perform [complex manipulation tasks](#) that defy explicit programming. It turns out that all of these advances fall under the umbrella of RL research. I also became interested in RL myself over the last ~year: I worked [through Richard Sutton's book](#), read through [David Silver's course](#), watched [John Schulmann's lectures](#), wrote an [RL library in Javascript](#), over the summer interned at DeepMind working in the DeepRL group, and most recently pitched in a little with the design/development of [OpenAI Gym](#), a new RL benchmarking toolkit. So I've certainly been on this funwagon for at least a year but until now I haven't gotten around to writing up a short post on why RL is a big deal, what it's about, how it all developed and where it might be going.



Examples of RL in the wild. **From left to right:** Deep Q Learning network playing ATARI, AlphaGo, Berkeley robot stacking Legos, physically-simulated quadruped leaping over terrain.

It's interesting to reflect on the nature of recent progress in RL. I broadly like to think about four separate factors that hold back AI:

1. Compute (the obvious one: Moore's Law, GPUs, ASICs),
2. Data (in a nice form, not just out there somewhere on the internet - e.g. ImageNet),
3. Algorithms (research and ideas, e.g. backprop, CNN, LSTM), and
4. Infrastructure (software under you - Linux, TCP/IP, Git, ROS, PR2, AWS, AMT, TensorFlow, etc.).

<http://karpathy.github.io/2016/05/31/rl/>

Components of the AlphaGo

The core parts of the Alpha Go comprise of:

- **Monte Carlo Tree Search:** AI chooses its next move using MCTS
- **Residual CNNs (Convolutional Neural Networks):** AI assesses new positions using these networks
- **Reinforcement learning:** Trains the AI by using the current best agent to play against itself

In this blog, we will **focus on the working of Monte Carlo Tree Search** only. This helps AlphaGo and AlphaGo Zero smartly explore and reach interesting/good states in a finite time period which in turn helps the AI reach human level performance.

It's application extends beyond games. MCTS can theoretically be applied to any domain that can be described in terms of $\{state, action\}$ pairs and simulation used to forecast outcomes. Don't worry if this sounds too complex right now, we'll break down all these concepts in this article.

<https://www.analyticsvidhya.com/blog/2019/01/monte-carlo-tree-search-introduction-algorithm-deepmind-alphago/>

<https://www.youtube.com/watch?v=0g9SIVdv1PY>

Proximal Policy Optimization

We're releasing a new class of reinforcement learning algorithms, Proximal Policy Optimization (PPO), which perform comparably or better than state-of-the-art approaches while being much simpler to implement and tune. PPO has become the default reinforcement learning algorithm at OpenAI because of its ease of use and good performance.



PPO lets us train AI policies in challenging environments, like the Roboschool one shown above where an agent tries to reach a target (the pink sphere), learning to walk, run, turn, use its momentum to recover from minor hits, and how to stand up from the ground when it is knocked over.

Policy gradient methods are fundamental to recent breakthroughs in using deep neural networks for control, from video games, to 3D locomotion, to Go. But getting good results via policy gradient methods is challenging because they are sensitive to the choice of stepsize — too small, and progress is hopelessly slow; too large and the signal is overwhelmed by the noise, or one might see catastrophic drops in performance. They also often have very poor sample efficiency, taking millions (or billions) of timesteps to learn simple tasks.

<https://openai.com/blog/openai-baselines-ppo/>

Why Dota?

We started OpenAI Five in order to work on a problem that felt outside of the reach of existing deep reinforcement learning^[1] algorithms. We hoped that by working on a problem that was unsolvable by current methods, we'd need to make a big increase in the capability of our tools. We were expecting to need sophisticated algorithmic ideas, such as hierarchical reinforcement learning, but we were surprised by what we found: the fundamental improvement we needed for this problem was scale. Achieving and utilizing that scale wasn't easy and was the bulk of our research effort!



OpenAI Five sees the world as a bunch of numbers that it must decipher. It uses the same general purpose learning code whether those numbers represent the state of a Dota game (about 20,000 numbers) or robotic hand (about 200).

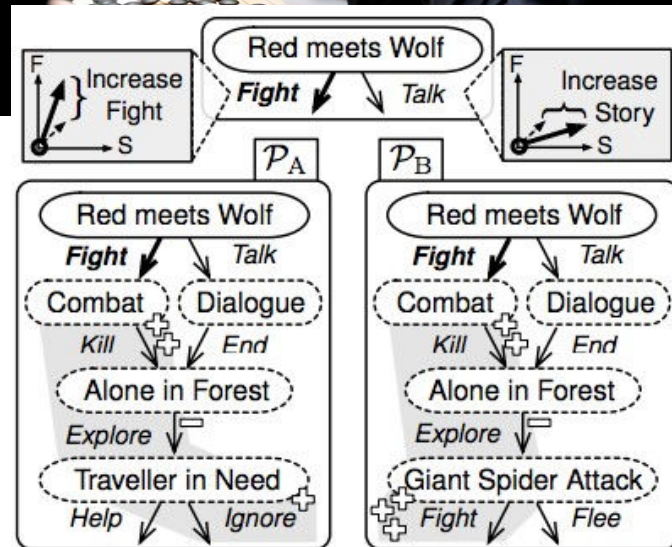
To build OpenAI Five, we created a system called Rapid which let us run PPO at previously unprecedented scale. The results exceeded our wildest expectations, and we produced a world class Dota bot without hitting any fundamental performance limits.

<https://openai.com/blog/how-to-train-your-openai-five/>

Reinforcement Learning in Games



廈門大學
XIAMEN UNIVERSITY



自强不息，止于至善

Reinforcement Learning: An Introduction

[Richard S. Sutton](#)
and [Andrew G. Barto](#)

Second Edition, in progress
MIT Press, Cambridge, MA, 2017

[Online draft](#) [New Code](#) [Solutions](#) [Course Materials](#)

16 Applications and Case Studies

- 16.1 TD-Gammon
- 16.2 Samuel's Checkers Player
- 16.3 Watson's Daily-Double Wagering
- 16.4 Optimizing Memory Control
- 16.5 Human-level Video Game Play
- 16.6 Mastering the Game of Go
 - 16.6.1 AlphaGo
 - 16.6.2 AlphaGo Zero
- 16.7 Personalized Web Services
- 16.8 Thermal Soaring

Scholarly articles for [sutton and barto reinforcement learning](#)

[Reinforcement learning](#): An introduction - [Sutton](#) - Cited by 25899

[Reinforcement learning](#) is direct adaptive optimal ... - [Sutton](#) - Cited by 348

<http://incompleteideas.net/book/the-book-2nd.html>

One-armed Bandit Problem



廈門大學
XIAMEN UNIVERSITY

Scholarly articles for [sutton](#) and [barto](#) reinforcement learning

Reinforcement learning: An introduction - [Sutton](#) - Cited by 25899

Reinforcement learning is direct adaptive optimal ... - [Sutton](#) - Cited by 348



自强不息，止于至善

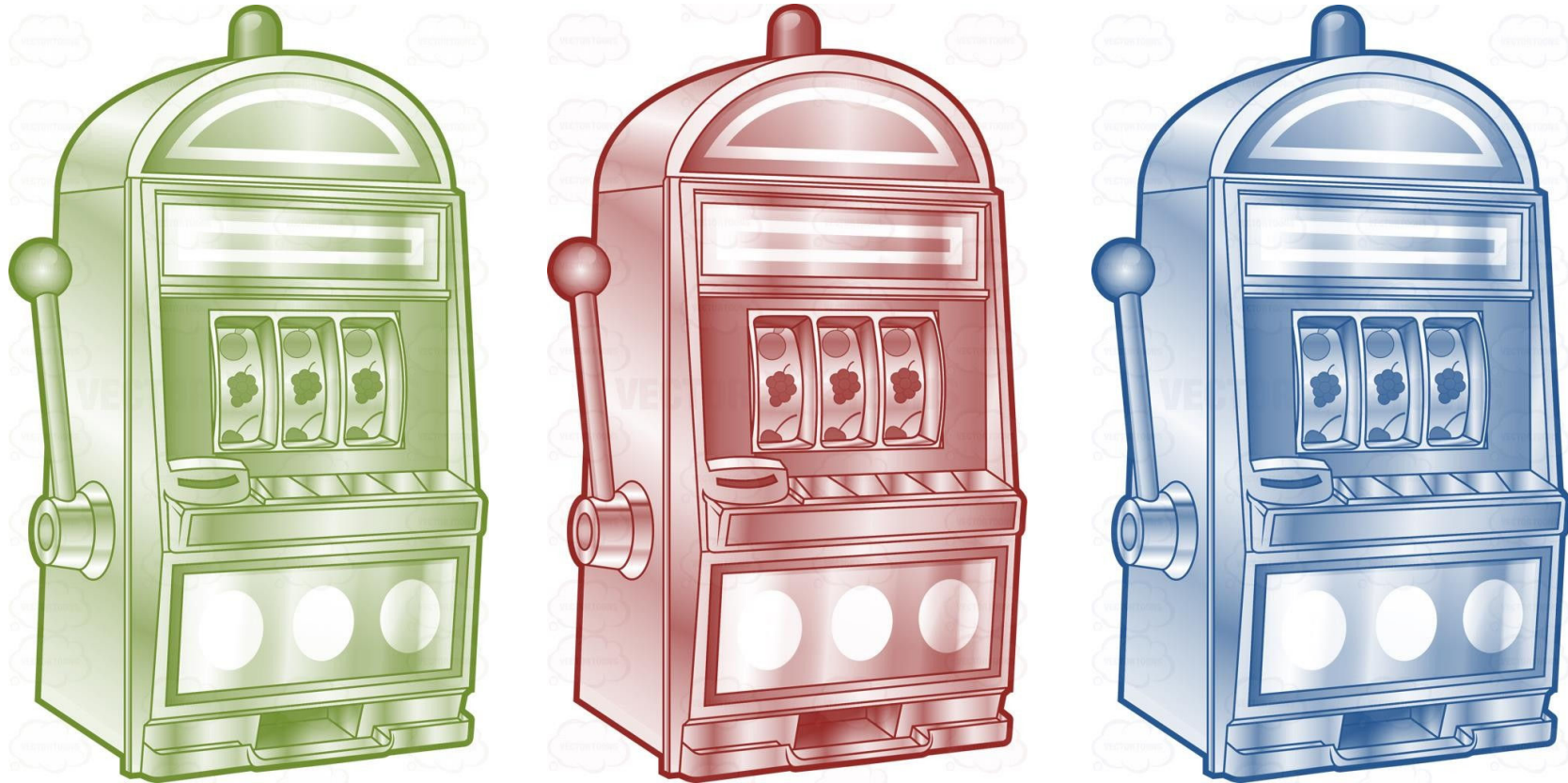
One-armed Bandit Process

Pull #	Response	Believed Probability of Jackpot
1	WIN (1.0)	1.0
2	LOSS (0.0)	0.5
3	LOSS (0.0)	0.33...
4	LOSS (0.0)	0.25
...	...	...
$N.$	LOSS (0.0)	0.1



Unknown True Probability

Multi-armed Bandit Problem

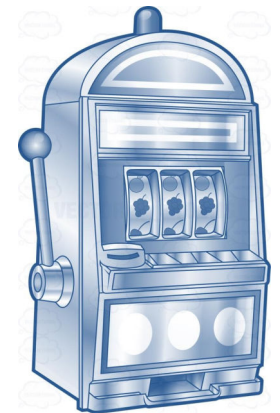
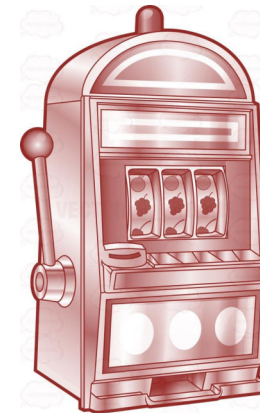
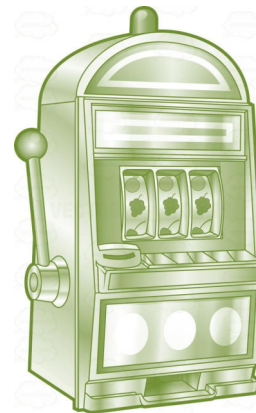


“Through repeated action selections you are to maximize your winnings by concentrating your actions on the best levers.” – Sutton & Barto

自强不息，止于至善

Now what?

- We can't just keep playing one bandit to figure out its potential for reward (money)
- Want to maximize reward across all bandits
- We need to trade off making money with current knowledge and gaining knowledge – Exploitation vs. Exploration
 - Exploitation vs. Exploration



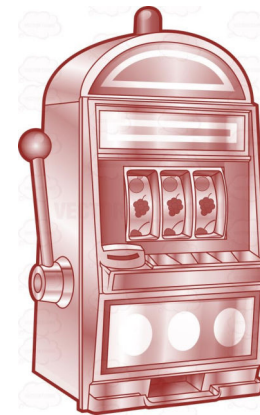
Just Exploitation

- Greedy Strategy where we always pick the machine we think will give us the best reward (reminder: machines give rewards according to a probability)

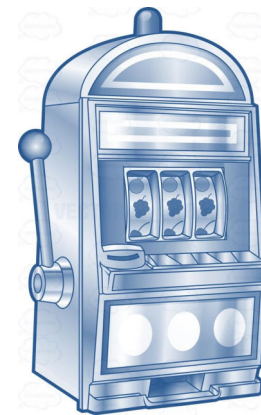
Machine	Response	Belief (G,R,B)
B	LOSS	(-, -, 0.0)
R	WIN	(0, 1, 0)
G	LOSS	(0, 1, 0)
R	LOSS	(0, 0.5, 0)
...	...	...



50%



25%



10%

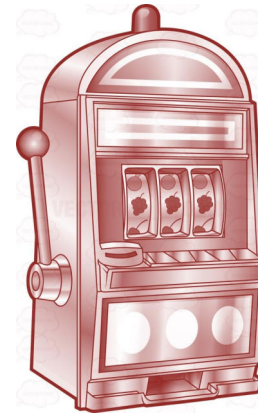
Just Exploration

- Always just pick a random machine, no matter what we believe.

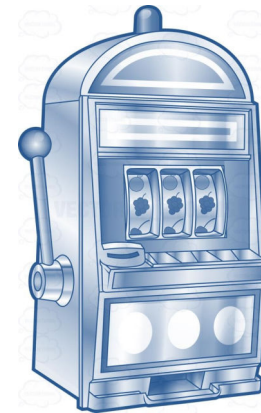
Machin e	Response	Belief (G,R,B)
B	LOSS	(-, -, 0.0)
R	WIN	(0, 1, 0)
G	LOSS	(0, 1, 0)
B	LOSS	(0, 1, 0)
...	...	...



50%



25%



10%

Epsilon (ϵ) Greedy (ϵ -Greedy)



- ($\epsilon=0.1$)
 - 90% of the time try the best machine according to our current beliefs
 - 10% of the time take random action

Now we can “escape” from early greedy mistakes!

Regret

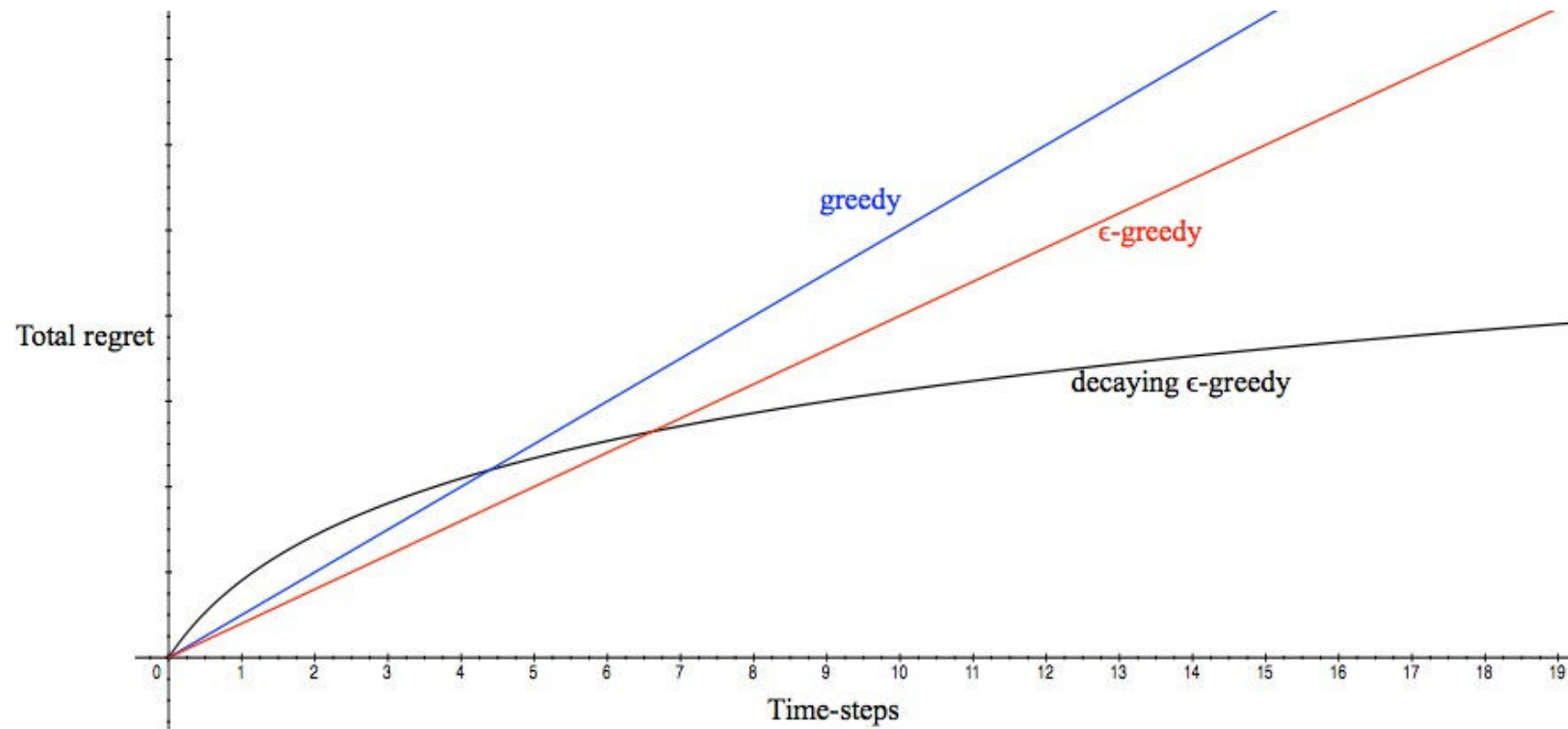


- Imagine a perfect agent, who always takes the best action
 - We can compare that agent's payout with our agent's payout for each action to get the regret for that step
 - Summing regret across all time steps gives us the total regret
-
- Maximize total reward == minimize total regret

Different Strategies and Regret



廈門大學
XIAMEN UNIVERSITY



自强不息，止于至善

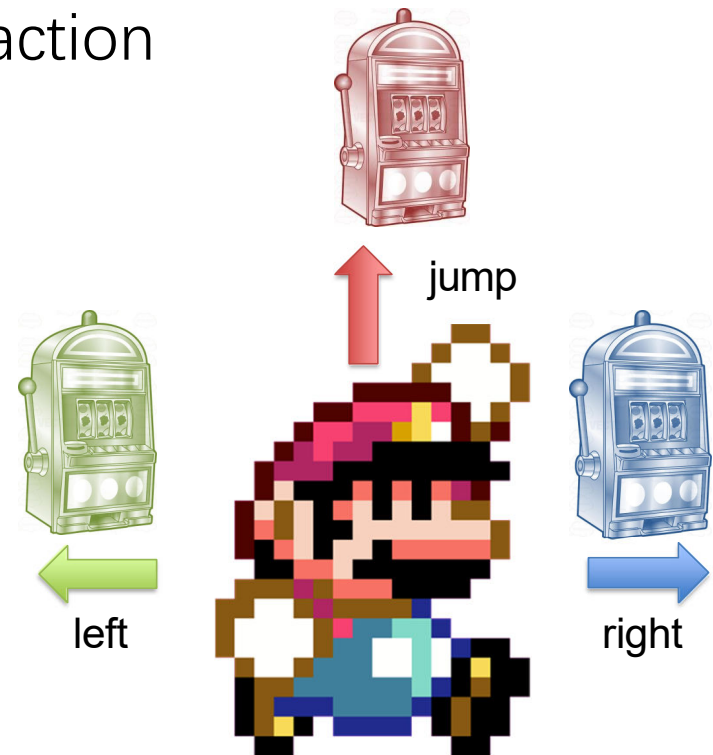
Decaying ϵ -Greedy



- Drop ϵ lower and lower according to some schedule
 - Now: logarithmic asymptotic regret
-
- Downside: we need to have enough domain knowledge to come up with a schedule beforehand

What does this have to do with games?

- We can imagine mapping this onto a video game domain
- Each distinct machine is now a distinct action
- Reward: Not dying, end of level, etc.



But these values will probably differ based on context...



廈門大學
XIAMEN UNIVERSITY



Value of going left versus right will be different if an enemy is to the left or the right.

State

- Multi-armed bandit problem is a “single state” or “stateless” **Markov Decision Process**
- But in games (and most cases we care about) there is more than one state



Andrey Markov

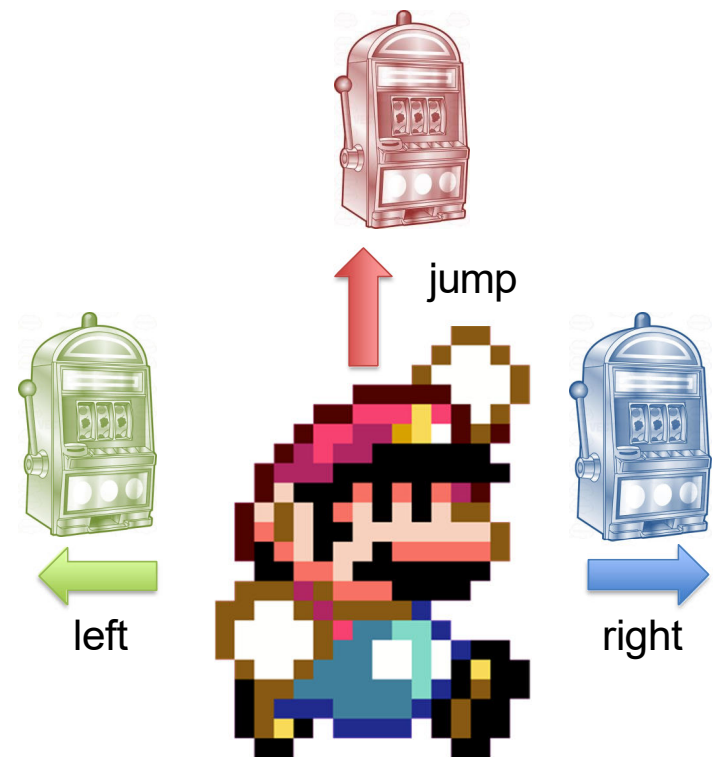
Markov Decision Process



- S : finite set of states
- A : finite set of actions
- $P(s, s_1)$: Probability that action a takes us from state s to state s_1
- $R(s, s_1)$: Reward for transitioning from state s to state s_1
- γ : Discount factor ($[0-1]$). Demonstrates the difference in importance between future awards and present awards

High-level Idea

- If the multi-armed bandit problem was a single state MDP, we can think of learning a strategy to play a game as solving this problem for every state of the game



Markovian State



- We call a state representation *Markovian* if it has all the information we need to make an optimal decision

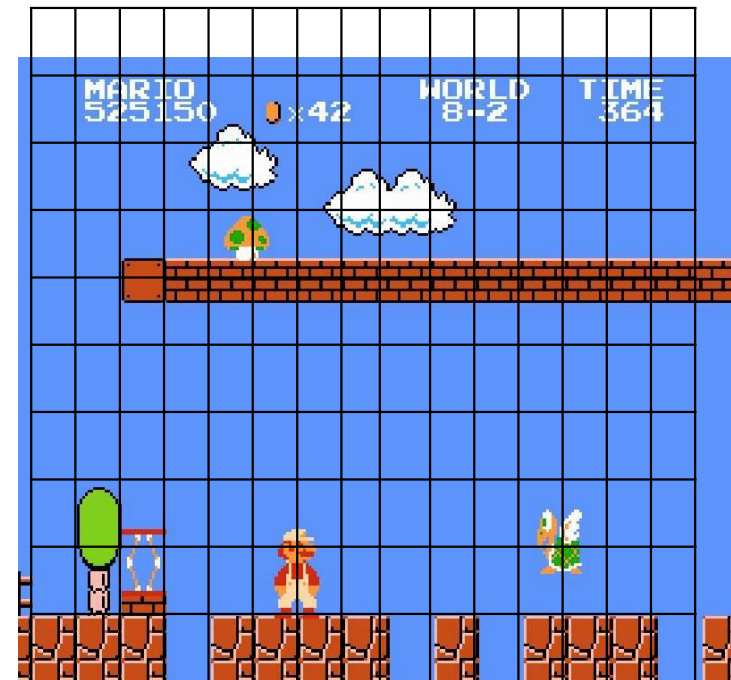
State Representation Examples

- List of facts
 - enemy to right
 - powerup above
 - fire mario
 - on ground
 - etc...



State Representation Examples

- Grid Representations
 - Segment all locations to tiles
 - All enemies represented individually? Or just as “enemy”
 - How much of the level to include in the state? Just screen?
 - Smaller? Larger?



State Representation Examples

- What pixels are present on the screen at this time?



State Representation Tradeoffs



- More complex state representations ensure that an agent has all info needed
- Less complex state representations speed up training (more states “look” the same)
- Goal: find the middle ground. Simplest state representation that still allows for optimal performance

Question



- What state representations would you use for the following? Why?
 - Tic-Tac-Toe
 - A simple platformer
 - A real time strategy game (Civilization/Starcraft)

Possible Answers



- Tic-Tac-Toe: The Gameboard
- Simple platformer: grid of screen width and height, with values for collideable elements, breakable elements, empty spaces, and enemy types (flying, etc)
- RTS: World map (?) + list of facts about currently running commands (building troops, moving troops, upgrading building, etc) (?)

Markov Decision Process



- S : finite set of states
- **A : finite set of actions**
 - – What are they in Tic-Tac-Toe? Platformers? RTS?
- $P(s, s_1)$: Probability that action a takes us from state s to state s_1
- $R(s, s_1)$: **Reward** for transitioning from state s to state s_1
- γ : Discount factor ($[0-1]$). Demonstrates the difference in importance between future awards and present awards
 - *What is the optimal action a to take for every state s ?*

Goal of the MDP



- Find the optimal action a to take for every state s
- We refer to this strategy as the policy
- Policy is represented as $\pi(s): S \rightarrow A$
 - What is the optimal action to take in state s ?

MDP: Reward Function



- Better understood as “feedback”
 - An MDP’s reward can be positive or negative
- How to give reward? And what reward to give?

MDP: Transition function ($P(s|s_1,a)$)



廈門大學
XIAMEN UNIVERSITY

- Also sometimes called a forward model
- Gives probability of transition from one state to another given a particular action
- Typically this is defined at a high level, not an individual state
 - E.g. When trying to move forward there is a 50% chance of moving forward, a 25% chance of moving to the left and a 25% chance of moving to the right
 - Not “when mario is at position (10,15) there is a 80% chance moving right gets mario to (11,15)”

Value Iteration



- Value Iteration:
 - every state has a value $V(s)$, generally represented as a table
 - Known transition probabilities
 - Known reward for every transition
- Algorithm
 - Start with arbitrary values for all $V(s)$
 - Every iteration we update the value of every state according to the rewards we can get from nearby states
 - Stop when values are no longer changing

Value Iteration Update



$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s,a) (R(s,a,s') + \gamma V_k(s'))$$

Value Iteration Update



$$Q_{k+1}(s,a) = \sum_{s'} P(s'|s,a) (R(s,a,s') + \gamma V_k(s'))$$

$$V_k(s) = \max_a Q_k(s,a)$$

^This probably looks like gibberish huh?

Value Iteration Update



$$Q_{k+1}(s,a) = \sum_{s'} P(s'|s,a) (R(s,a,s') + \gamma V_k(s'))$$

$$V_k(s) = \max_a Q_k(s,a)$$

Translation:

The value of a particular action a in state s =

- the sum across all neighboring states of the
- the probability of going from state s to state s' given action a multiplied by
- The reward of entering state s' by action a from state s plus the discount factor γ multiplied by the current value of state s'

Value Iteration Update



$$V_k(s) = \max_a Q_k(s, a)$$

The new value of state s is then

- the max value within $Q_k(s, a)$

Value Iteration Update



assign $V_0[S]$ arbitrarily $k \leftarrow 0$

repeat

$k \leftarrow k+1$

for each state s **do**:

$$Q_{k+1}(s,a) = \sum_{s'} P(s'|s,a) (R(s,a,s') + \gamma V_k(s')) \quad V_{k+1}(s) = \max_a Q_{k+1}(s,a)$$

until $V_k(s) - V_{k+1}(s) < \theta$:

Get Policy From Value Table



- Remember we want the optimal action a for each state s
- $\pi[s] = \operatorname{argmax}_a \sum_{s'} P(s'|s,a) (R(s,a,s') + \gamma V_k[s'])$
Translation:
- Take the value of a that gives the max value...
- For each neighboring state and action a
 - Multiply the transition probability from state s to state s' with action a by
 - The sum of
 - the reward for entering s' from s with a and
 - The product of the discount value by the Value table for s'

Question



- Give me all the pieces needed to run an MDP for the simple game of your choice
 - S is the set of all states
 - A is the set of all actions
 - P is state transition function specifying $P(s'|s,a)$
 - R is a reward function $R(s,a,s')$

Pros/Cons



- Pros
 - Once value iteration is done running we have an agent that can act optimally in any state very, very quickly (table lookup)
- Cons:
 - Once value iteration is done running
 - What if we don't have a transition function? IE we don't know probability of getting from s to s' ?
 - What if we have a black box that allows us to simulate it...

Seems like we have some extra stuff we don't really need huh?

$$Q_{k+1}(s,a) = \sum_{s'} P(s'|s,a) (R(s,a,s') + \gamma V_k(s'))$$

$$V_k(s) = \max_a Q_k(s,a)$$

Seems like we have some extra stuff we don't really need huh?

$$Q_{k+1}(s,a) = \sum_{s'} P(s'|s,a) (R(s,a,s') + \gamma V_k(s'))$$
$$V_k(s) = \max_a Q_k(s,a)$$

Traditionally, there was a preference for keeping track of $V(s)$ over $Q(s,a)$ as it was smaller

Q-table Update

$$Q[s,a] \leftarrow Q[s,a] + \alpha * (r + \gamma * \max_{a'} Q[s',a'] - Q[s,a])$$

A Q-table enumerates all possible states and all possible actions and gives the utility of each action in each state

States	Action 1	Action 2	Action 3	Action 4	Action 5
s1	0.1	0.5	0.9	0.4	0.0
s2	0.8	0.2	0.1	0.0	-1.0
s3	0.0	0.0	0.0	0.0	0.0
...	...	...	...	...	...

Q-learning algorithm



Assign $Q[S,A]$ arbitrarily

observe current state s

repeat

select and carry out an action a

observe reward r and state s'

$$Q[s,a] \leftarrow Q[s,a] + \alpha * (r + \gamma * \max_{a'} Q[s',a'] - Q[s,a])$$

until termination

the learned action-value function, Q , directly approximates the optimal action-value function, independent of the policy being followed

learning rate, α , ranges $(0,1]$

1 if env is deterministic
often 0.1

discount factor, γ , ranges $[0,1]$

0 is myopic, 1 long-term

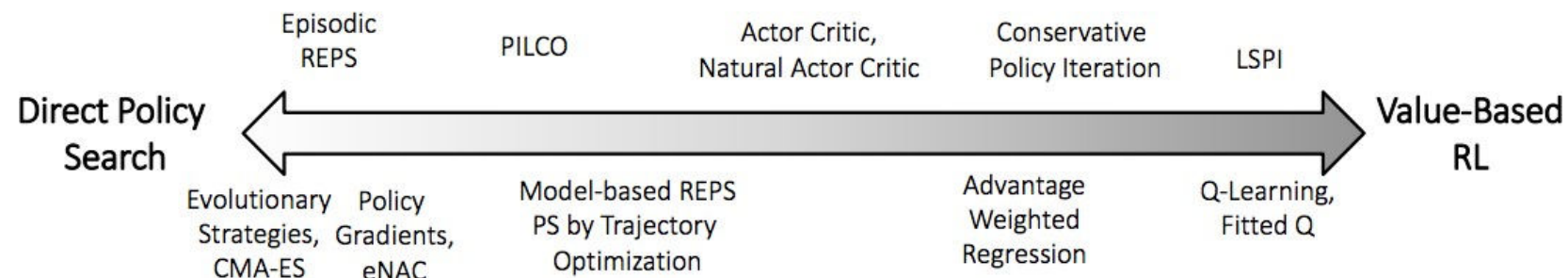
Q Learning



- Pros
 - We don't need to have a forward model/transition function
 - Means we can go into an environment blind
- Cons
 - Takes up more memory than Value Iteration as we have to hold a $|S|*|A|$ table instead of just an $|S|$ -sized table
 - Can be ameliorated with function approximation
 - Bigger table means (at times) longer to converge

Q Learning

- The most general game playing/decision making technique we've seen so far
- Only needs a reward function, list of actions, and state representation
- But that state space sure is too massive to be useful huh?







廈門大學
XIAMEN UNIVERSITY

感谢您的观看
Thank you for watching